Run & Demo Guide

How to start, show, test and explain the running system

For the professor and the team. It starts from a computer where OpenDoor is already installed (the installation steps are in the repository README) and shows how to run every service, which accounts to use, how to present the system, and – click by click – what each screen sends to the backend and what changes in the database.

Live system	https://opendoor-rust.vercel.app
API gateway	`https://gateway-production-4e58.up.railway.app`
Walkthrough	Captured October 19, 2026 on a local, freshly seeded stack
Version	October 19, 2026 · commit `7442849`

OpenDoor – PWD job matching by workplace accommodations · SIA 1 and IPT final project · Group 3 – ICARUS, IT31S1

Contents

1 About this guide

2 The system at a glance

3 Starting and stopping

4 Addresses and demo accounts

5 Presentation demo script (about 10 minutes)

6 UI ↔ backend walkthrough
- 6.1 Log in (candidate)
- 6.2 Find jobs that match my needs
- 6.3 Open a job and its Workplace Accessibility Snapshot (mashup)
- 6.4 Apply for the job
- 6.5 Open the application (marks it reviewed)
- 6.6 Mark the applicant as hired
- 6.7 Check the employer with the business registry (SOAP)
- 6.8 Approve the employer
- 6.9 Approve a job posting

7 Validation script

8 Testing tools

9 Troubleshooting

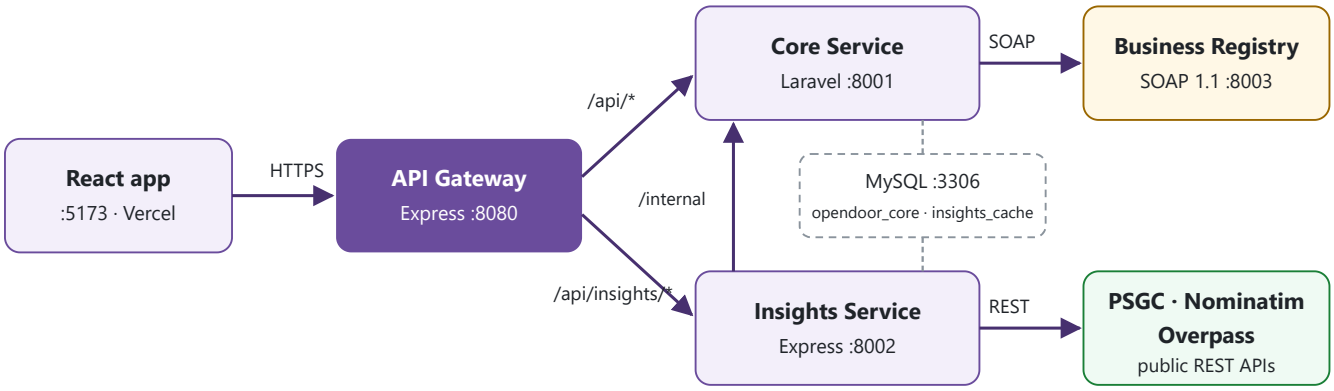
A Rebuilding this guide

1 About this guide

Everything needed to run OpenDoor on a prepared computer, present it, and explain what happens between the screen and the database.

- **Not covered:** installing PHP, Node.js, MySQL, the packages, the `.env` files and the databases. Those steps are in the repository `README.md`; this guide assumes they are done.
- **Sections 3–5** are for running and presenting. **Section 6** follows nine clicks through the whole system with the real requests, responses, log lines and database rows, captured from the running system.
- **Secrets:** passwords, tokens, the service key and the SOAP credentials are never printed. Where one appears it is replaced by `••••••  (hidden)`.
- **Production:** React on Vercel (https://opendoor-rust.vercel.app); gateway, services and MySQL on Railway (Singapore). Only the gateway is public.

2 The system at a glance



- The browser talks **only** to the gateway. The gateway adds an `X-Request-ID`, applies CORS and the rate limit, and forwards `/api/insights/*` to Insights and every other `/api/*` path to Core.
- Core owns the main data (`opendoor_core`) and calls the SOAP registry when an admin checks an employer.
- Insights builds the Workplace Accessibility Snapshot. It reads posting data from Core's `/internal` route with a service key (never from Core's database) and caches the public APIs in its own `insights_cache` database.
- Every error from any service has the same JSON envelope: `{"error":{"code","message","request_id"}}`.

Component	Folder	Port
MySQL 8	Windows service `MySQL80`	3306
Business Registry (SOAP)	`services/business-registry-soap`	8003
Core Service (Laravel)	`services/core-api`	8001
Insights Service	`services/insights-service`	8002
API Gateway	`gateway`	8080
React app (Vite)	`frontend`	5173

3 Starting and stopping

3.1 Before you start

- MySQL 8 is running (Windows service `MySQL80`).
- Every service folder has its `.env` file and the databases are migrated and seeded (installation, see `README.md`).
- Open one PowerShell terminal per service (VS Code: **Terminal** → **Split** works well, so all logs are visible during the demo). Each command runs from the repository folder `C:\dev\opendoor`.

3.2 Start order and commands

Start them top to bottom – each one needs the ones above it. Leave every terminal open: it is the service, and it shows its log.

#	Component	Terminal: go to	Start command	Ready when you see
0	MySQL 8	–	`Start-Service MySQL80` (admin PowerShell) – usually starts with Windows	`Get-Service MySQL80 → Running`
1	Business Registry (SOAP)	`cd services/business-registry-soap`	`php -S 127.0.0.1:8003 -t public`	`Development Server (http://127.0.0.1:8003) started`
2	Core Service (Laravel)	`cd services/core-api`	`php artisan serve --host=127.0.0.1 --port=8001`	`Server running on [http://127.0.0.1:8001]`
3	Insights Service	`cd services/insights-service`	`npm run dev`	`OpenDoor Insights Service listening on http://localhost:8002 …`
4	API Gateway	`cd gateway`	`npm run dev`	`OpenDoor gateway listening on http://localhost:8080 (core: …, insights: …)`
5	React app (Vite)	`cd frontend`	`npm run dev`	`Local: http://localhost:5173/`

Use `127.0.0.1` for the PHP servers: on Windows `localhost` may bind IPv6 only, and the other services call `127.0.0.1`. Open the app itself at `http://localhost:5173` – that is the origin the gateway allows (CORS).

3.3 Check that everything runs

- Browser: `http://localhost:8080/api/health` → `{"status":"ok","services":{"core":"up","insights":"up"}}`.
- Or run the validation script (section 7): `python tests/validate/validate_opendoor.py` → `Result: 14/14 checks passed`.
- Then open `http://localhost:5173` and log in with a demo account (section 4).

3.4 Stopping, and freeing a busy port

- Stop a service with **Ctrl+C** in its terminal (answer **Y** if asked). Stop in reverse order: React, gateway, Insights, Core, registry. MySQL can keep running.
- If a start command says the port is in use, find the process that holds it and stop it (PowerShell):

```
Get-NetTCPConnection -LocalPort 8001 -State Listen | Select-Object LocalPort, OwningProcess
Get-Process -Id <OwningProcess>          # check what it is (php, node)
Stop-Process -Id <OwningProcess>
```

Replace 8001 with the busy port (8001 Core, 8002 Insights, 8003 registry, 8080 gateway, 5173 React).

3.5 Optional: background jobs

Forgot-password emails are queued so the response time never reveals whether an account exists. Locally, send them with `php artisan queue:work database --stop-when-empty` in `services/core-api`; with the local `log` mailer the email (and its link) is written to `storage/logs/laravel.log`. In production the scheduler inside the Core container works the queue every minute and mail goes out through Brevo.

3.6 Production

- App: https://opendoor-rust.vercel.app · Gateway health: `https://gateway-production-4e58.up.railway.app/api/health`.
- Nothing needs to be started. A few minutes before presenting, open the health URL (wakes the services) and open each job page once (fills the snapshot cache).
- Logs: Railway dashboard → project → service → **Deployments** → **View logs** (the gateway shows the same JSON lines as locally).

4 Addresses and demo accounts

	Local	Production
App (React)	`http://localhost:5173`	https://opendoor-rust.vercel.app
API gateway	`http://localhost:8080/api`	`https://gateway-production-4e58.up.railway.app/api`
Health check	`http://localhost:8080/api/health`	`https://gateway-production-4e58.up.railway.app/api/health`
Core / Insights	`http://127.0.0.1:8001` / `http://127.0.0.1:8002`	private (Railway network only)
SOAP registry WSDL	`http://127.0.0.1:8003/soap?wsdl`	private (Railway network only)

Demo accounts

Email	Name	Role	Use it to show
`admin@opendoor.test`	OpenDoor Admin	Admin	Employer verification with the SOAP registry, posting approval, users, reports, feedback moderation, master data, analytics
`candidate@opendoor.test`	Maria Santos	Candidate	Search with "Matches my needs", job details + Accessibility Snapshot, apply, saved jobs, applications
`candidate2@opendoor.test`	Juan Dela Cruz	Candidate	A hired candidate: the accommodation feedback form (trust score input)
`employer@opendoor.test`	Ana Reyes	Employer (Bayanihan Tech Solutions, verified)	Postings, applicants, status changes (reviewed → hired), published trust score
`employer2@opendoor.test`	Ramon Villanueva	Employer (Kalinga Foods Inc., waiting for verification on a fresh seed)	An employer the admin still has to verify; a posting waiting for approval

> **Passwords:** all five accounts share one demo password (`DEMO_PASSWORD`, set when the database is seeded). It is not written in this copy – ask the team for the copy with passwords or for the password itself.

New candidate or employer accounts can be registered from the home page; they see a short welcome tour and a “Get started” checklist. Admin accounts are only seeded.

5 Presentation demo script (about 10 minutes)

One path through all three roles that touches every component: gateway, Core, the mashup, the SOAP registry and the database. Section 6 shows what each of these clicks does behind the screen.

Time	As	Do	Say
0:00	Guest	Open the home page, then **Find jobs**. Point at the utility bar (text size, high contrast, quiet mode).	OpenDoor matches persons with disabilities to jobs by the accommodations a workplace gives, not by disability type.
1:00	Candidate	Log in as `candidate@`. Dashboard → **Find jobs**, tick **Matches my needs**.	The candidate saved her needs once; Core ranks every open posting by how many of them it meets.
2:30	Candidate	Open **Data Encoder** → **Show Workplace Accessibility Snapshot**.	This is the mashup: the Insights Service combines Core's posting, PSGC place names, Nominatim coordinates and OpenStreetMap places from Overpass. If a source is down the page still works.
3:30	Candidate	Write a cover note and an interview request → **Send application**.	One request through the gateway; Core stores the application and notifies the employer.
4:30	Employer	Log out, log in as `employer@`. Dashboard → the posting → **Applicants** → open Maria's application → **Mark as hired**.	Opening the application marks it reviewed; hiring records the promised accommodations that Maria will rate after 30 days.
6:00	Admin	Log in as `admin@`. **Employers** → Kalinga Foods Inc. → **Check with the business registry** → **Approve**.	Core calls the legacy SOAP registry (WS-Security) and stores its answer. The admin decides; the registry only advises.
7:30	Admin	**Postings** → **Approve** Packing Line Staff.	Only approved postings are visible to candidates.
8:00	Anyone	Open the employer's public page (trust score card) and **How trust scores work**.	Hired employees rate each promised accommodation anonymously; the score shows after 3 responses.
8:30	Presenter	Show DevTools → Network on one click (request, `X-Request-ID`), and the gateway terminal line with the same id.	Every request goes through the gateway; the id follows it through every service and the error envelope.
9:30	Presenter	Run `python tests/validate/validate_opendoor.py` (or show the CI run on GitHub).	14 checks: every service is up, the gateway routes correctly and Insights reaches Core with its service key.

Before the presentation

- Production: open `/api/health` and every open job page once (wakes Railway, fills the snapshot cache).
- Local: start everything (section 3) and run the validation script.
- Production: as the admin, check that an employer and a posting are still waiting for approval (**Employers** / **Postings**). If earlier tests already approved Kalinga Foods or Packing Line Staff, use any waiting one, or register a new employer and submit a posting before the presentation.
- The demo changes data (an application, a hire, an approval). On production these stay; locally re-seed with `php artisan migrate:fresh --seed` to start again (deletes all local data).
- Keep a browser with DevTools open (F12 → **Network**, filter **Fetch/XHR**) and the gateway terminal visible.

6 UI ↔ backend walkthrough

Nine clicks from the demo, each with the screen, the HTTP call the page made, the log lines in the service terminals and the database rows before and after.

How to show this live

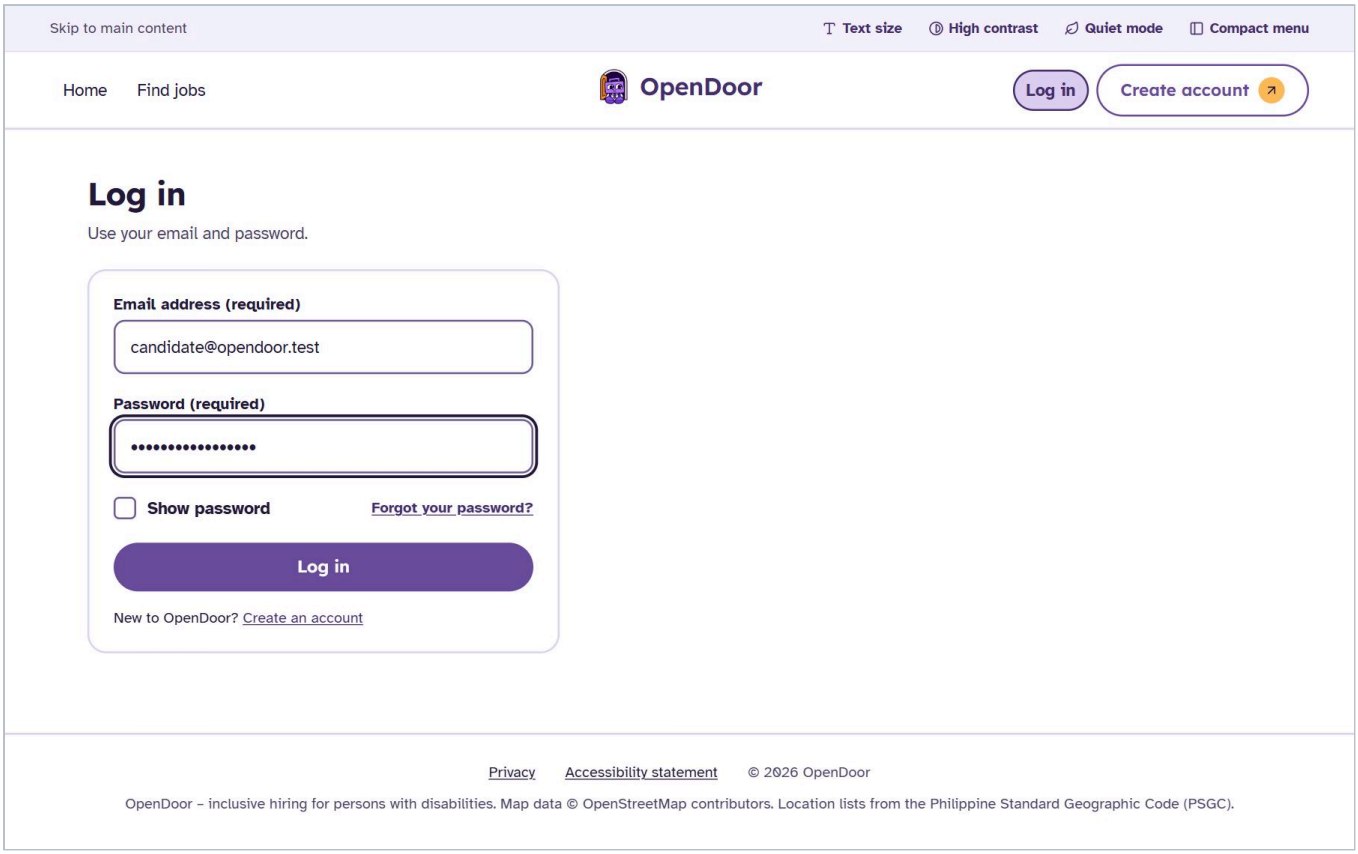
- **Browser – DevTools:** F12 → **Network** → filter **Fetch/XHR**. Click a request: **Headers** (method, URL, status, `X-Request-ID`, `Authorization`), **Payload** (the JSON sent) and **Response** (the JSON returned). The `OPTIONS` requests are CORS preflights answered by the gateway.
- **Gateway terminal:** one JSON line per request with the same `request_id` as the response header – search for it to follow one click.
- **Core / registry terminals:** each request Core and the SOAP registry receive. Insights logs every request and every external source that fails.
- **Database:** MySQL Workbench or `mysql -u opendoor_core -p opendoor_core`, then run the `SELECT` shown under each step.

Captured October 19, 2026 on the local stack, freshly seeded database (`opendoor_core_test`), built React app (`npm run build` + `npx vite preview --port 5173`), so each call appears once. Times in the logs are UTC (Manila = UTC+8). Core here ran on PHP's built-in server; `php artisan serve` shows the same requests in its own format. Response bodies are shortened (long lists show 2 items).

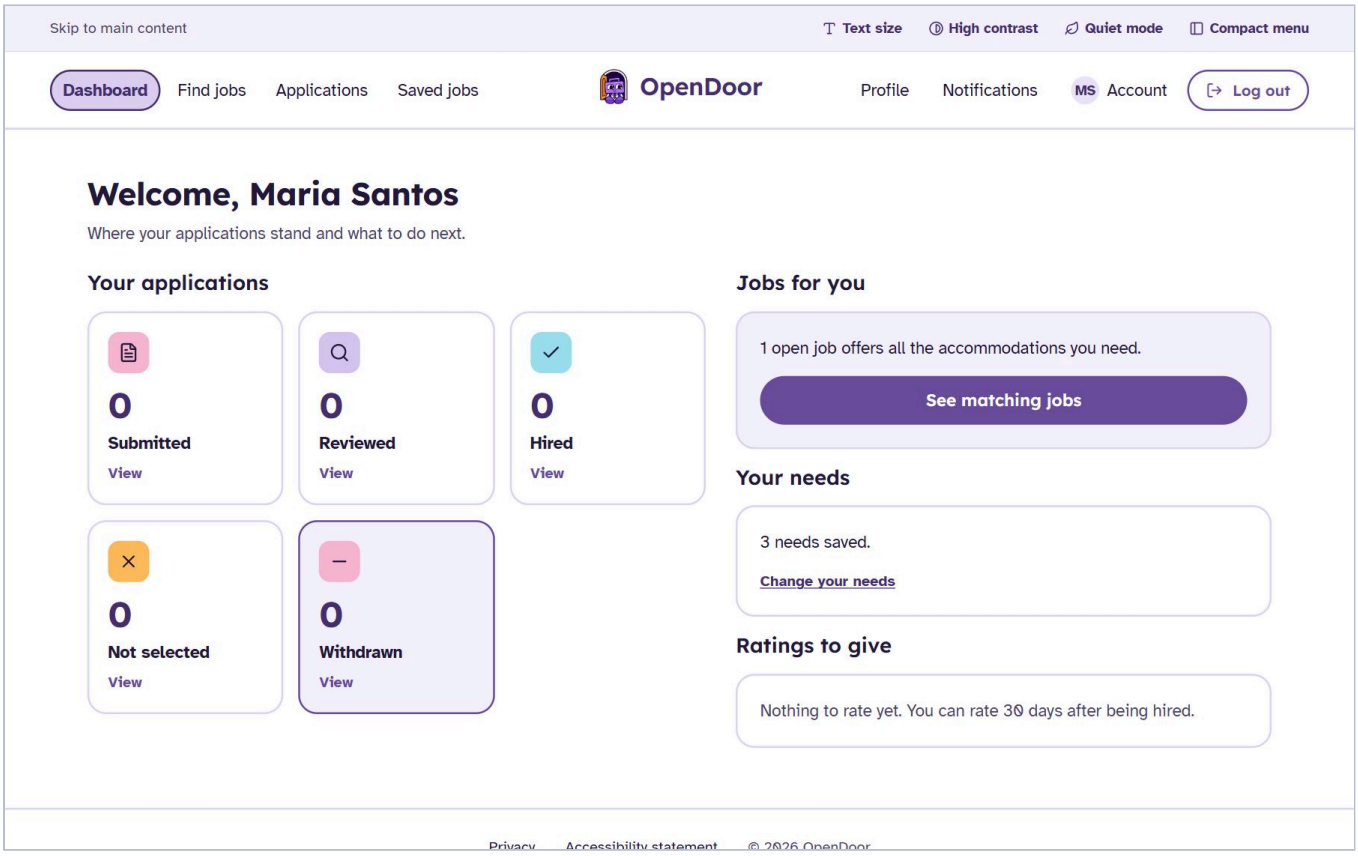
6.1 Log in (candidate)

Candidate – candidate@opendoor.test

In the UI: On **Log in**, type the email and password and press **Log in**.



The login form, filled in



After Log in: the candidate dashboard

WHAT HAPPENS BEHIND THE SCREEN

- React (`LoginPage` → `AuthContext`) sends `POST /api/login` to the gateway – the only backend address the browser knows (`VITE_API_URL`).
- The gateway adds an `X-Request-ID`, checks CORS and the rate limit (120 requests per minute per IP) and forwards `/api/*` to Core unchanged.
- Core checks the password hash and that the account is not suspended, then creates a Sanctum token (one row in `personal_access_tokens`, stored as a SHA-256 hash).
- The browser keeps the token and sends it as `Authorization: Bearer …` on every later call; the dashboard loads with `GET /api/candidate/dashboard`.

REQUEST (DEVTOOLS → NETWORK → HEADERS / PAYLOAD)

```
POST http://localhost:8080/api/login
Status: 200
X-Request-ID: eddfccfa-17ec-4805-8346-ed4d59aa6046

{
  "email": "candidate@opendoor.test",
  "password": "•••••• (hidden)"
}
```

RESPONSE (DEVTOOLS → RESPONSE)

```
{
  "token": "1|•••••• (hidden)",
  "user": {
    "id": 2,
    "name": "Maria Santos",
    "email": "candidate@opendoor.test",
    "role": "candidate",
    "is_active": true,
    "display_prefs": null,
    "email_notifications": true,
    "onboarding": {
      "tour_done": true,
      "checklist_hidden": true
    }
  }
}
```

Also on this screen: `GET /api/notifications`, `GET /api/candidate/dashboard`, `GET /api/candidate/profile` (page data: user, notifications, lists).

SERVICE LOGS (THE TERMINALS)

Gateway terminal (one JSON line per request, no headers or bodies)

```
{"time":"2026-10-19T15:39:03.297Z","request_id":"eddfccfa-17ec-4805-8346-ed4d59aa6046","method":"POST","path":"/api/login","stat
us":200,"duration_ms":1249}
```

Core terminal

```
[POST] URI: /api/login
```

DATABASE

A new token row for the candidate (user 2). The table holds only a hash of the token – the plain token is shown to the browser once.

Before

```
SELECT id, tokenable_id, name, last_used_at, created_at FROM personal_access_tokens WHERE tokenable_id = 2 ORDER BY id DESC
LIMIT 1
```

id	tokenable_id	name	last_used_at	created_at
(no rows)				

After

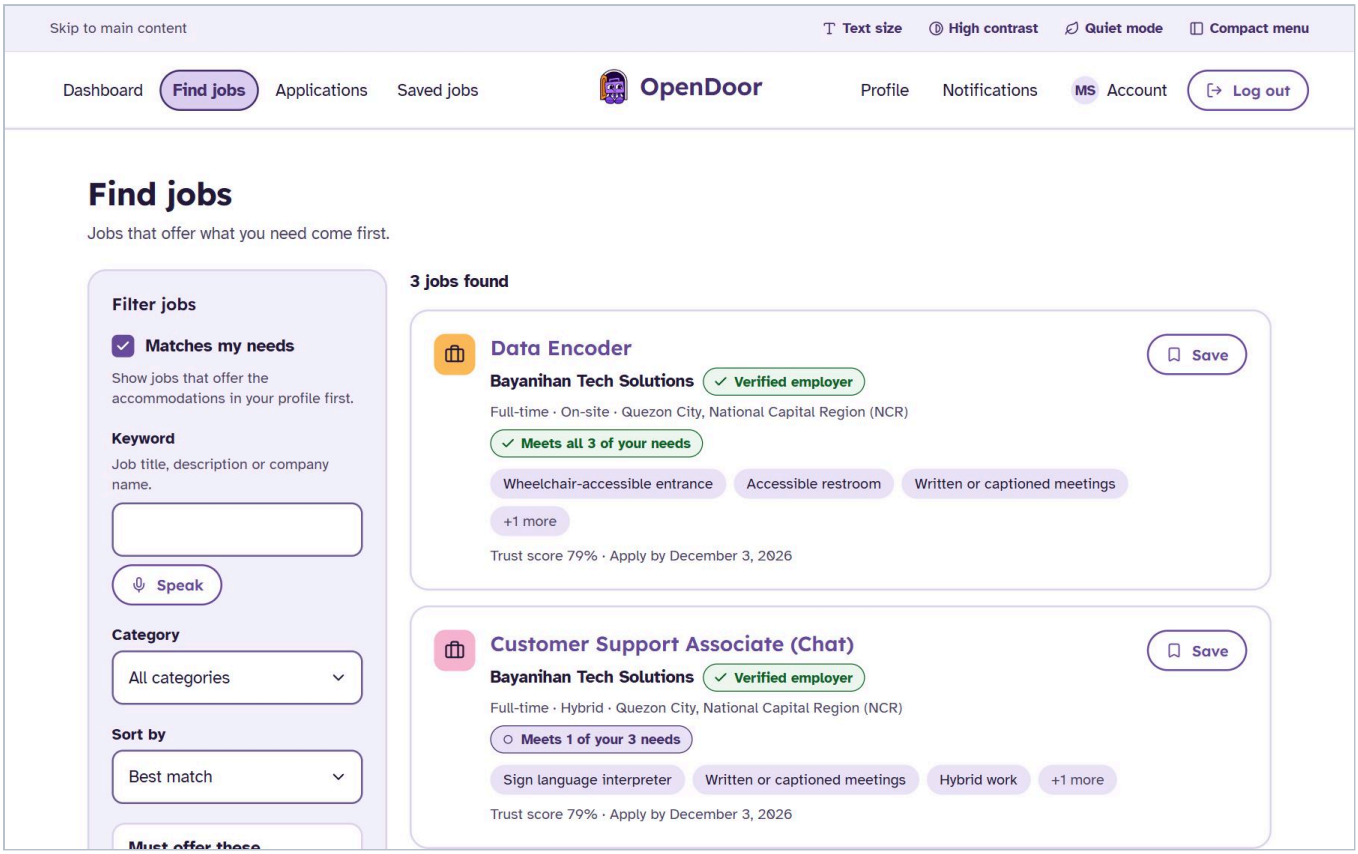
```
SELECT id, tokenable_id, name, last_used_at, created_at FROM personal_access_tokens WHERE tokenable_id = 2 ORDER BY id DESC LIMIT 1
```

id	tokenable_id	name	last_used_at	created_at
1	2	api	2026-10-19 23:39:03	2026-10-19 23:39:03

6.2 Find jobs that match my needs

Candidate

In the UI: Open **Find jobs** and tick **Matches my needs**.



Find jobs with “Matches my needs” ticked

WHAT HAPPENS BEHIND THE SCREEN

- `GET /api/job-postings?match=1&sort=best` – Core reads the candidate’s saved needs and ranks the open, approved postings by how many of them each one offers – full matches first, then newest.
- Each posting comes back with `match: { met, total, missing }`; the disability type is never used for matching (RA 10173).
- With `npm run dev` React’s StrictMode loads each page twice, so DevTools shows every call two times. The built app (Vercel, and these captures) calls once.

REQUEST (DEVTOOLS → NETWORK → HEADERS / PAYLOAD)

```
GET http://localhost:8080/api/job-postings?match=1&sort=best
Status: 200
X-Request-ID: bd8d8710-6698-400b-988f-5a6166379191
Authorization: Bearer 1|•••••• (hidden)
```

RESPONSE (DEVTOOLS → RESPONSE)


```
{
  "data": [
    {
      "id": 1,
      "title": "Data Encoder",
      "description": "Encode and check customer records in our CRM.\nMonday to Friday, 9:00 to 17:00. Training is prov
ided.",
      "employer": {
        "id": 1,
        "company_name": "Bayanihan Tech Solutions",
        "verified": true
      },
```

```
      "category": {
        "id": 1,
        "name": "Administrative"
      },
      "address": "25 Scout Tobias St., South Triangle",
      "location": {
        "psgc_code": "1381300000",
        "label": "Quezon City, National Capital Region (NCR)"
      },
      "employment_type": "full_time",
      "work_setup": "onsite",
      "interview_format": "online",
      "status": "open",
      "closes_at": "2026-12-03",
      "accommodations": [
        {
          "id": 1,
          "name": "Wheelchair-accessible entrance",
          "group": "physical_access",
          "description": "Step-free entrance with a ramp or level access.",
          "is_retired": false,
          "note": "Ramp at the main entrance on Scout Tobias St."
        },
… (121 more lines)
```


Also on this screen: `GET /api/me`, `GET /api/notifications`, `GET /api/categories`, `GET /api/accommodations` (page data: user, notifications, lists).

SERVICE LOGS (THE TERMINALS)

Gateway terminal (one JSON line per request, no headers or bodies)


```
{"time":"2026-10-19T15:39:04.723Z","request_id":"bd8d8710-6698-400b-988f-5a6166379191","method":"GET","path":"/api/job-posting
s","status":200,"duration_ms":195}
```


Core terminal

```
[GET] URI: /api/job-postings
```

DATABASE

Nothing changes – this is a read. The rows below are the candidate's needs that the match uses (`candidate_accommodation`).

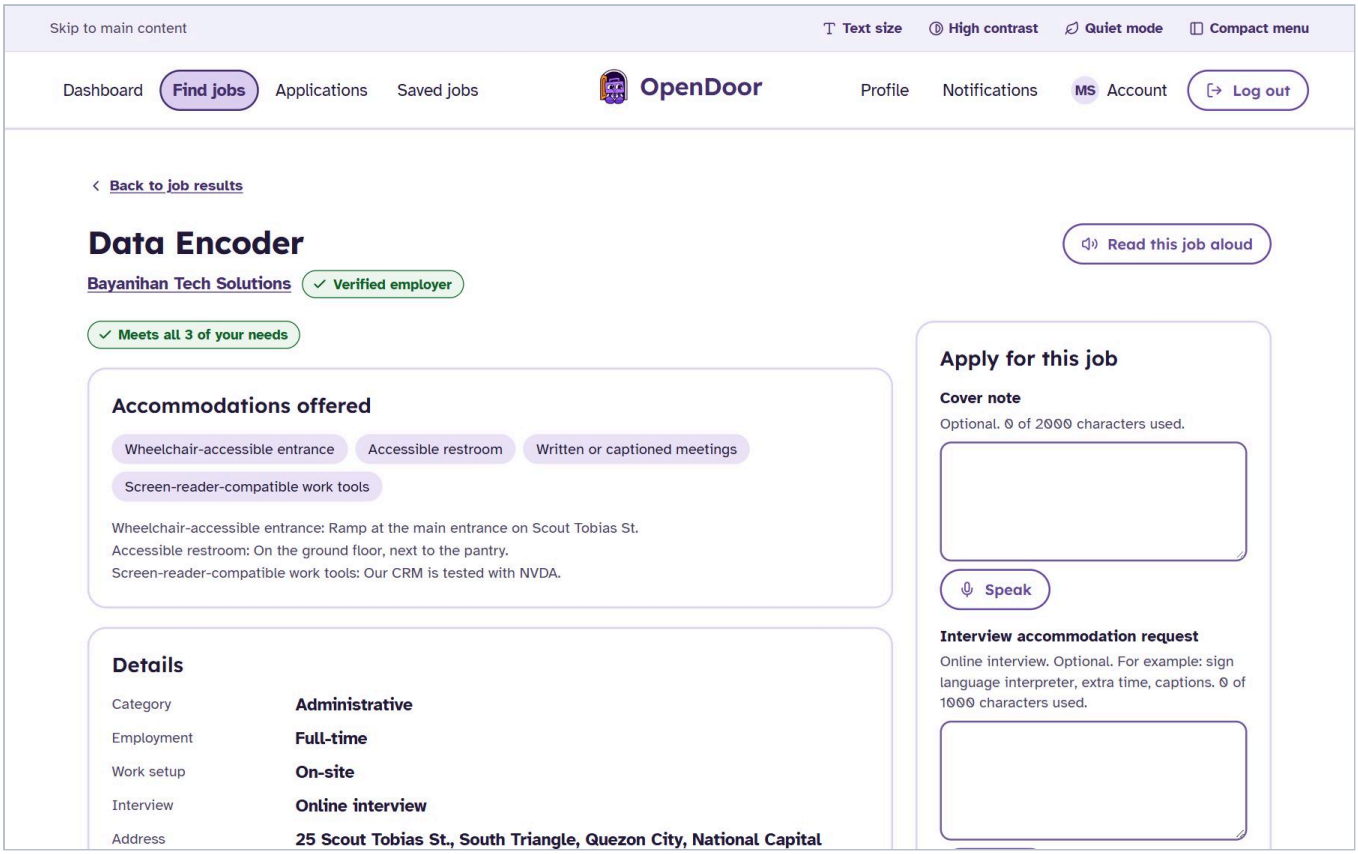
Read by this step

```
SELECT a.id, a.name FROM candidate_accommodation ca JOIN accommodations a ON a.id = ca.accommodation_id WHERE
ca.candidate_profile_id = 1 ORDER BY a.id
```

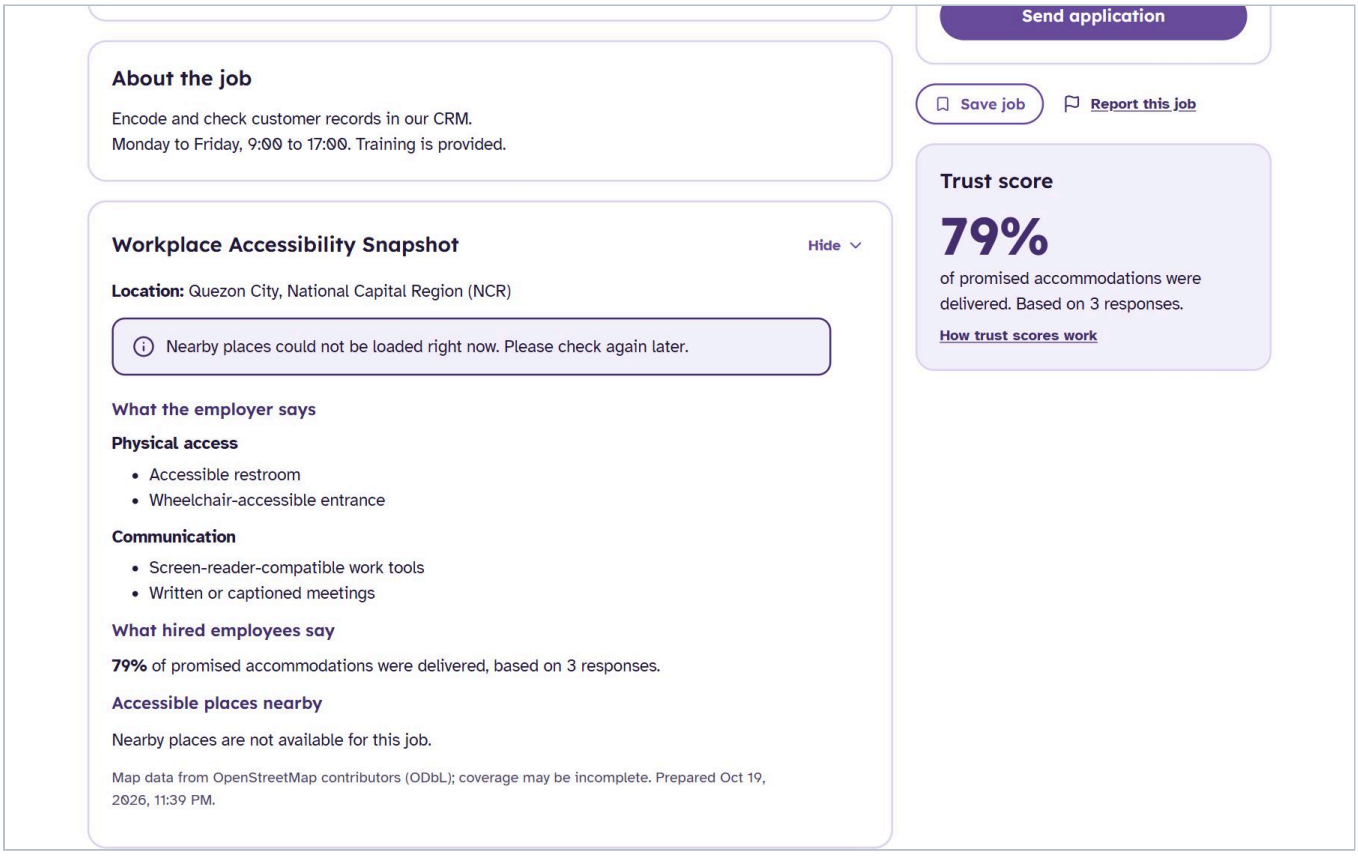
id	name
1	Wheelchair-accessible entrance
2	Accessible restroom
6	Written or captioned meetings

6.3 Open a job and its Workplace Accessibility Snapshot (mashup) Candidate

In the UI: Open **Data Encoder** and press **Show Workplace Accessibility Snapshot**.



Job details



The Workplace Accessibility Snapshot opened

WHAT HAPPENS BEHIND THE SCREEN

- The gateway sends `/api/insights/*` to the Insights Service (15 s timeout) instead of Core.
- Insights asks Core for the posting over the private route `GET /internal/job-postings/{id}/summary` with its `X-Service-Key` (service-to-service; the gateway never exposes `/internal`).
- Then PSGC (official place names), Nominatim (coordinates of the address) and Overpass (wheelchair-accessible places nearby). Each answer is cached in the `insights_cache` database.
- The whole snapshot must finish in 12 s. A source that is down or slow is reported in `sources` as `unavailable` or `stale` and the snapshot is still 200 – external services never block hiring.

REQUEST (DEVTOOLS → NETWORK → HEADERS / PAYLOAD)

```
GET http://localhost:8080/api/insights/job-postings/1
Status: 200
X-Request-ID: 345eb321-2148-4f43-9a06-104fb73b6e90
Authorization: Bearer 1|•••••• (hidden)
```

RESPONSE (DEVTOOLS → RESPONSE)


```
{
  "job_posting_id": 1,
  "location": {
    "psgc_code": "1381300000",
    "label": "Quezon City, National Capital Region (NCR)",
    "lat": 14.636788,
    "lon": 121.031402,
    "precision": "street"
  },
  "employer_claims": {
    "physical_access": [
      "Accessible restroom",
      "Wheelchair-accessible entrance"
    ],
    "communication": [
      "Screen-reader-compatible work tools",
      "Written or captioned meetings"
    ]
  },
  "community_verification": {
    "trust_score": 79,
    "based_on": 3,
    "breakdown": [
      {
        "accommodation_id": 2,
        "name": "Accessible restroom",
        "delivered": 2,
        "partial": 0,
        "not_delivered": 0,
        "of": 2
      },
      {
        "accommodation_id": 10,
        "name": "Flexible hours",
… (18 more lines)
```


Also on this screen: `GET /api/me`, `GET /api/notifications`, `GET /api/job-postings/1` (page data: user, notifications, lists).

SERVICE LOGS (THE TERMINALS)

Gateway terminal (one JSON line per request, no headers or bodies)


```
{"time":"2026-10-19T15:39:19.161Z","request_id":"345eb321-2148-4f43-9a06-104fb73b6e90","method":"GET","path":"/api/insights/job-postings/1","status":200,"duration_ms":12027}
```


Core terminal

```
[GET] URI: /internal/job-postings/1/summary
```

Insights terminal

{"time":"2026-10-19T15:39:19.159Z","request_id":"345eb321-2148-4f43-9a06-104fb73b6e90","method":"GET","path":"/api/insights/job-postings/1","status":200,"duration_ms":12008}

DATABASE

Insights owns its own cache tables (database `insights_cache`); Core's data is never copied there – Insights only asks Core's API. The rows below are the cached answers this posting's snapshot uses: the coordinates of its address (Nominatim, kept 30 days), the accessible places around them (Overpass, 24 h – after that the old copy is only used as `stale` when Overpass fails) and the PSGC place lists. `fetched_at` shows when each answer was fetched.

After – geocode

```
SELECT LEFT(address_hash, 12) AS address_hash, query, lat, lon, `precision`, fetched_at FROM geocode_cache WHERE query LIKE '25 Scout Tobias St., South Triangle%'
```

address_hash	query	lat	lon	precision	fetched_at
779d34899d3a	25 Scout Tobias St., South Triangle, Quezon City, Metro Manila	14.636788	121.031402	street	2026-10-06 23:49:17.056

After – overpass

```
SELECT `key`, JSON_LENGTH(payload) AS osm_elements, fetched_at FROM overpass_cache WHERE `key` LIKE '14.6368,121.0314,%'
```

key	osm_elements	fetched_at
14.6368,121.0314,500	50	2026-10-13 21:17:32.137

After – psgc

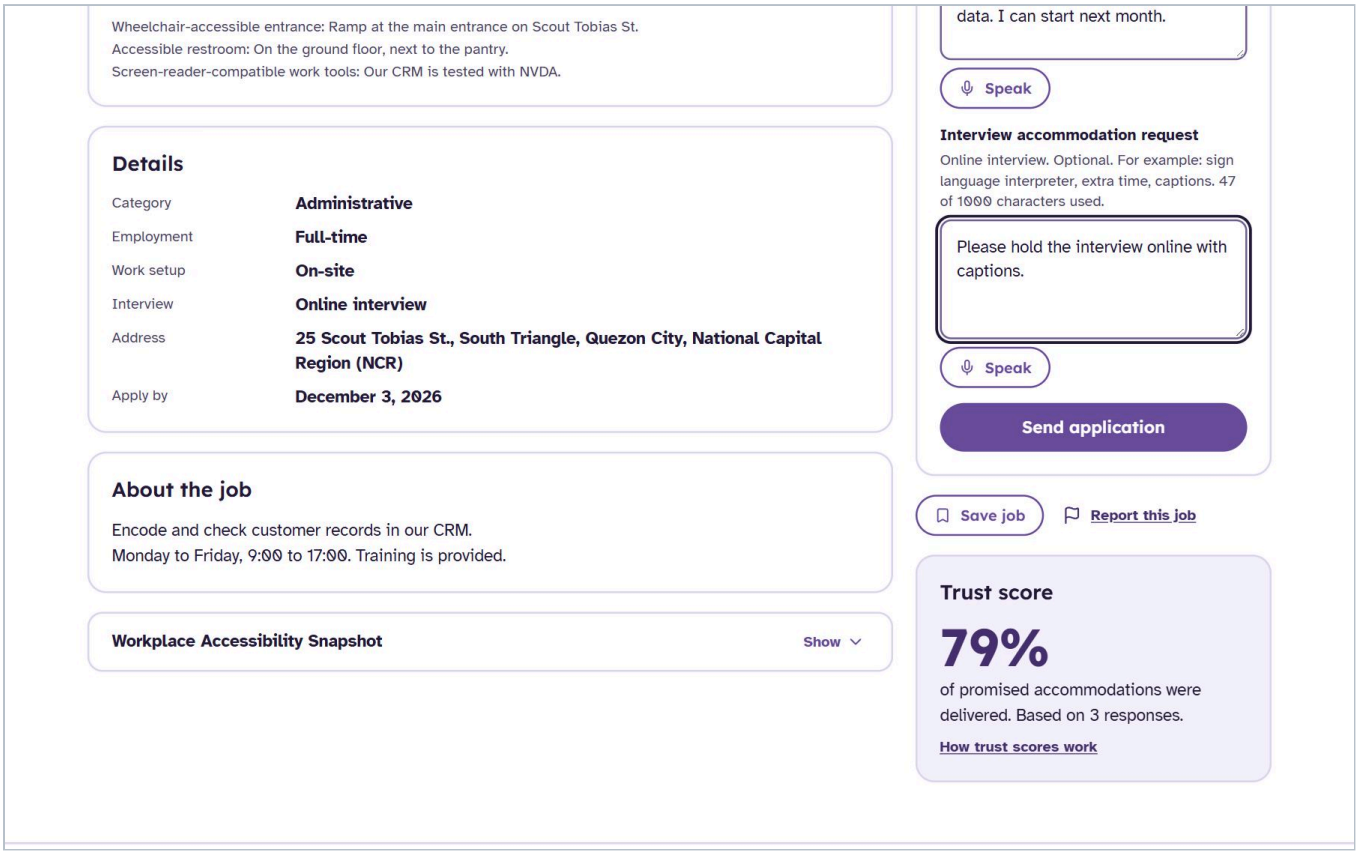
```
SELECT `key`, JSON_LENGTH(payload) AS entries, fetched_at FROM psgc_cache WHERE `key` LIKE '%1300000000' ORDER BY `key`
```

key	entries	fetched_at
cities:1300000000	17	2026-10-13 21:15:29.635
provinces:1300000000	0	2026-10-19 15:22:34.899

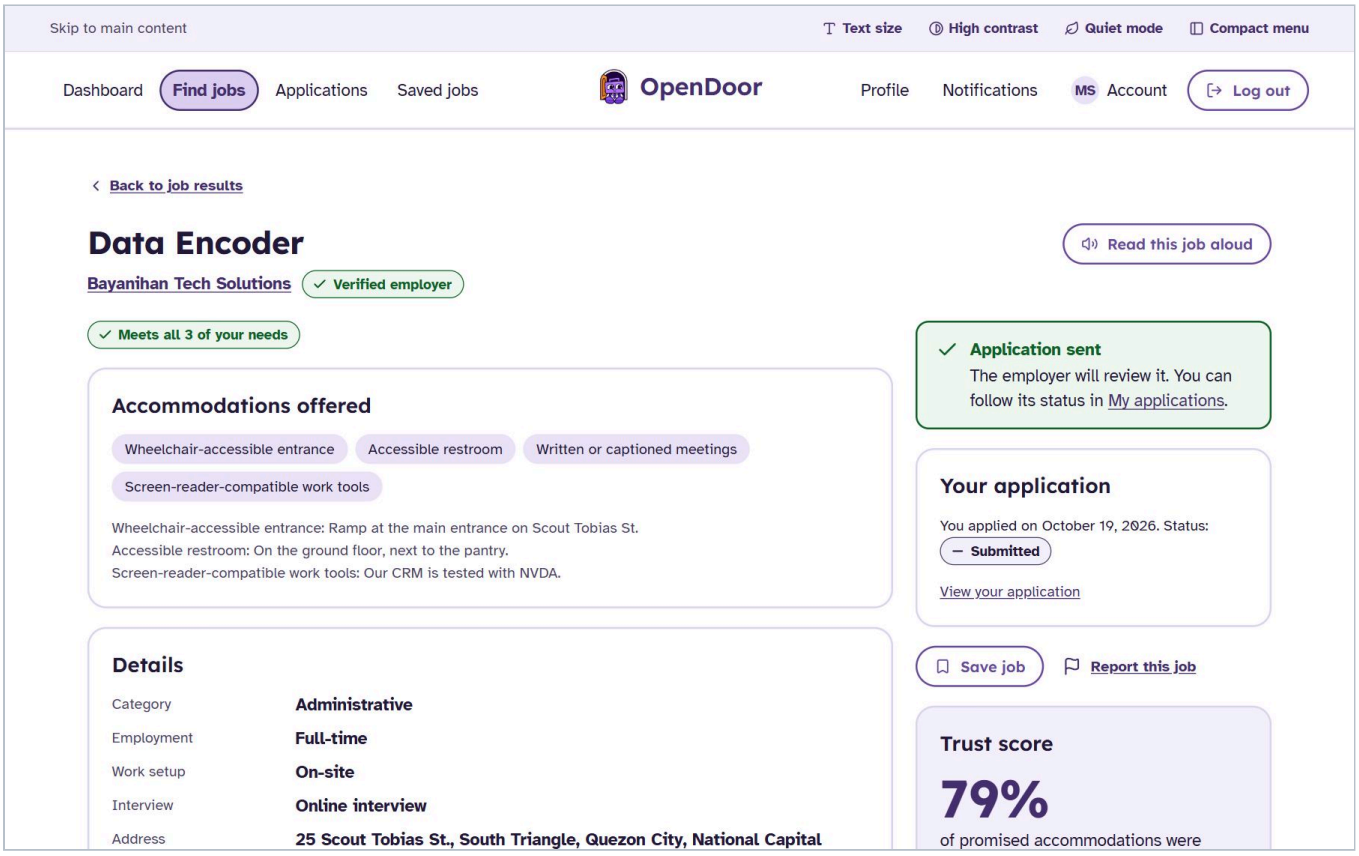
6.4 Apply for the job

Candidate

In the UI: Fill **Cover note** and **Interview accommodation request**, then press **Send application**.



The application form, filled in



After Send application

WHAT HAPPENS BEHIND THE SCREEN

- Core checks the role (candidate), that the posting is open and not past its closing date, and that the candidate has not applied already (409 `ALREADY_APPLIED`).
- It stores the application with status `pending` and notifies the employer (in-app notice, plus an email when the employer has email notifications on).

REQUEST (DEVTOOLS → NETWORK → HEADERS / PAYLOAD)

```
POST http://localhost:8080/api/job-postings/1/applications
Status: 201
X-Request-ID: a8c966ce-322f-4476-a16d-f402ad536fb4
Authorization: Bearer 1|•••••• (hidden)

{
  "cover_note": "I type 60 words per minute with a screen reader and I am careful with data. I can start next month.",
  "interview_request": "Please hold the interview online with captions."
}
```

RESPONSE (DEVTOOLS → RESPONSE)


```
{
  "id": 7,
  "job_posting_id": 1,
  "job_posting": {
    "id": 1,
    "title": "Data Encoder",
    "description": "Encode and check customer records in our CRM.\nMonday to Friday, 9:00 to 17:00. Training is provid
ed.",
    "employer": {
      "id": 1,
      "company_name": "Bayanihan Tech Solutions",
      "verified": true
    },
    "category": {
      "id": 1,
      "name": "Administrative"
    },
    "address": "25 Scout Tobias St., South Triangle",
    "location": {
      "psgc_code": "1381300000",
      "label": "Quezon City, National Capital Region (NCR)"
    },
    "employment_type": "full_time",
    "work_setup": "onsite",
    "interview_format": "online",
    "status": "open",
    "closes_at": "2026-12-03",
    "accommodations": [
      {
        "id": 1,
        "name": "Wheelchair-accessible entrance",
        "group": "physical_access",
        "description": "Step-free entrance with a ramp or level access.",
        "is_retired": false,
        "note": "Ramp at the main entrance on Scout Tobias St."
… (30 more lines)
```


Also on this screen: `GET /api/me`, `GET /api/notifications`, `GET /api/job-postings/1` (page data: user, notifications, lists).

SERVICE LOGS (THE TERMINALS)

Gateway terminal (one JSON line per request, no headers or bodies)


```
{"time":"2026-10-19T15:39:21.638Z","request_id":"a8c966ce-322f-4476-a16d-f402ad536fb4","method":"POST","path":"/api/job-posting
s/1/applications","status":201,"duration_ms":478}
```


Core terminal

```
[POST] URI: /api/job-postings/1/applications
```

DATABASE

A new `applications` row (status `pending`) and an `application_received` notification for the employer's account.

Before

```
SELECT id, job_posting_id, candidate_profile_id, status, reviewed_at, hired_at, hire_confirmed_by, created_at FROM applications WHERE candidate_profile_id = 1 AND job_posting_id = 1
```

id	job_posting_id	candidate_profile_id	status	reviewed_at	hired_at	hire_confirmed_by	created_at
(no rows)							

After – application

```
SELECT id, job_posting_id, candidate_profile_id, status, reviewed_at, hired_at, hire_confirmed_by, created_at FROM applications WHERE candidate_profile_id = 1 AND job_posting_id = 1
```

id	job_posting_id	candidate_profile_id	status	reviewed_at	hired_at	hire_confirmed_by	created_at
7	1	1	pending	NULL	NULL	NULL	2026-10-19 23:39:21

After – notifications

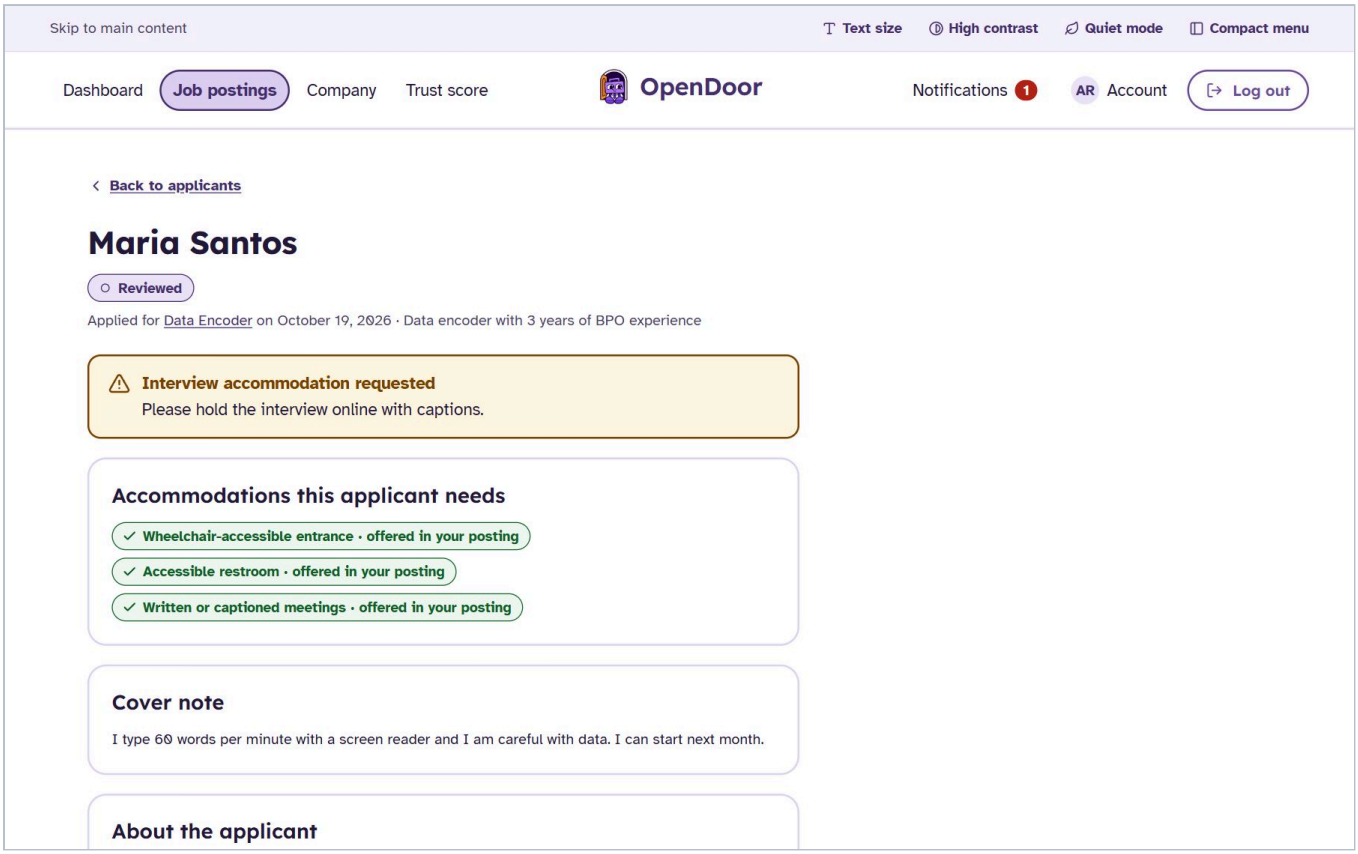
```
SELECT notifiable_id AS user_id, type, JSON_UNQUOTE(JSON_EXTRACT(data, '$.message')) AS message, created_at FROM notifications ORDER BY created_at DESC LIMIT 1
```

user_id	type	message	created_at
4	application_received	Maria Santos applied for Data Encoder.	2026-10-19 23:39:21

6.5 Open the application (marks it reviewed)

Employer – employer@opendoor.test

In the UI: As the employer, open the application (Dashboard → posting → **Applicants** → the applicant).



The employer's view of the application

WHAT HAPPENS BEHIND THE SCREEN

- Core's policy allows the employer to see only applications to their own postings (others get 403). Only through this application does the employer see the candidate's needs.
- The first time the employer opens a `pending` application Core sets it to `reviewed` and tells the candidate.

REQUEST (DEVTOOLS → NETWORK → HEADERS / PAYLOAD)

```
GET http://localhost:8080/api/applications/7
Status: 200
X-Request-ID: 9f9f441e-b466-4d83-93b2-38626830de26
Authorization: Bearer 2|•••••• (hidden)
```

RESPONSE (DEVTOOLS → RESPONSE)

```
{
  "id": 7,
  "job_posting_id": 1,
  "job_posting": {
    "id": 1,
    "title": "Data Encoder",
    "description": "Encode and check customer records in our CRM.\nMonday to Friday, 9:00 to 17:00. Training is provid
ed.",
    "employer": {
      "id": 1,
      "company_name": "Bayanihan Tech Solutions",
      "verified": true
    },
    "category": {
      "id": 1,
      "name": "Administrative"
    },
```

```
    "address": "25 Scout Tobias St., South Triangle",
    "location": {
      "psgc_code": "1381300000",
      "label": "Quezon City, National Capital Region (NCR)"
    },
    "employment_type": "full_time",
    "work_setup": "onsite",
    "interview_format": "online",
    "status": "open",
    "closes_at": "2026-12-03",
    "accommodations": [
      {
        "id": 1,
        "name": "Wheelchair-accessible entrance",
        "group": "physical_access",
        "description": "Step-free entrance with a ramp or level access.",
        "is_retired": false,
        "note": "Ramp at the main entrance on Scout Tobias St."
… (59 more lines)
```


Also on this screen: `GET /api/notifications`, `GET /api/me` (page data: user, notifications, lists).

SERVICE LOGS (THE TERMINALS)

Gateway terminal (one JSON line per request, no headers or bodies)


```
{"time":"2026-10-19T15:39:24.619Z","request_id":"9f9f441e-b466-4d83-93b2-38626830de26","method":"GET","path":"/api/applications/7","status":200,"duration_ms":201}
```


Core terminal

```
[GET] URI: /api/applications/7
```

DATABASE

Status `pending → reviewed`, `reviewed_at` filled, and a notification for the candidate.

Before

```
SELECT id, job_posting_id, candidate_profile_id, status, reviewed_at, hired_at, hire_confirmed_by, created_at FROM applications WHERE candidate_profile_id = 1 AND job_posting_id = 1
```

id	job_posting_id	candidate_profile_id	status	reviewed_at	hired_at	hire_confirmed_by	created_at
7	1	1	pending	NULL	NULL	NULL	2026-10-19 23:39:21

After – application

```
SELECT id, job_posting_id, candidate_profile_id, status, reviewed_at, hired_at, hire_confirmed_by, created_at FROM applications WHERE candidate_profile_id = 1 AND job_posting_id = 1
```

id	job_posting_id	candidate_profile_id	status	reviewed_at	hired_at	hire_confirmed_by	created_at
7	1	1	reviewed	2026-10-19 23:39:24	NULL	NULL	2026-10-19 23:39:21

After – notifications

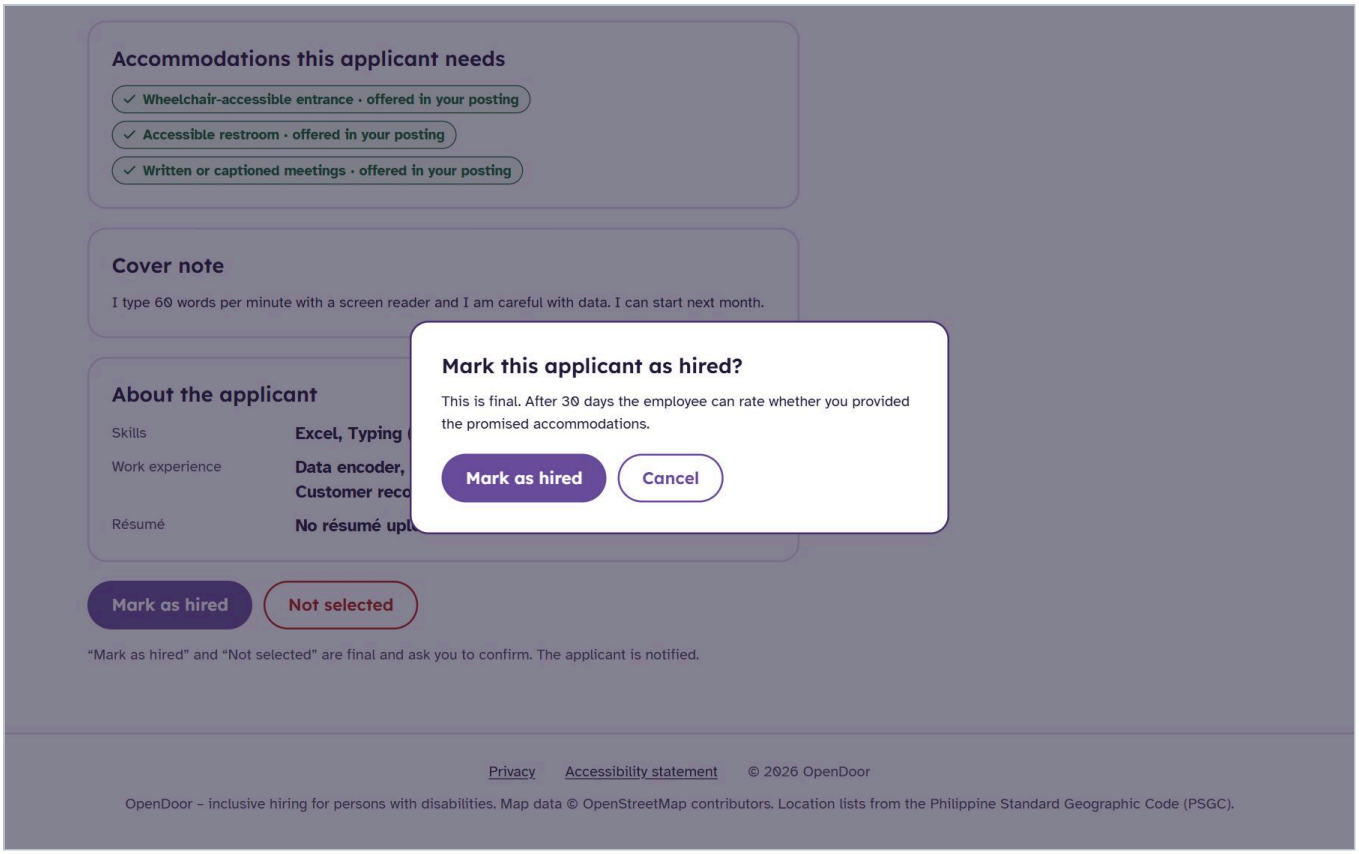
```
SELECT notifiable_id AS user_id, type, JSON_UNQUOTE(JSON_EXTRACT(data, '$.message')) AS message, created_at FROM notifications ORDER BY created_at DESC LIMIT 1
```

user_id	type	message	created_at
2	application_status_changed	Bayanihan Tech Solutions is reviewing your application for Data Encoder.	2026-10-19 23:39:24

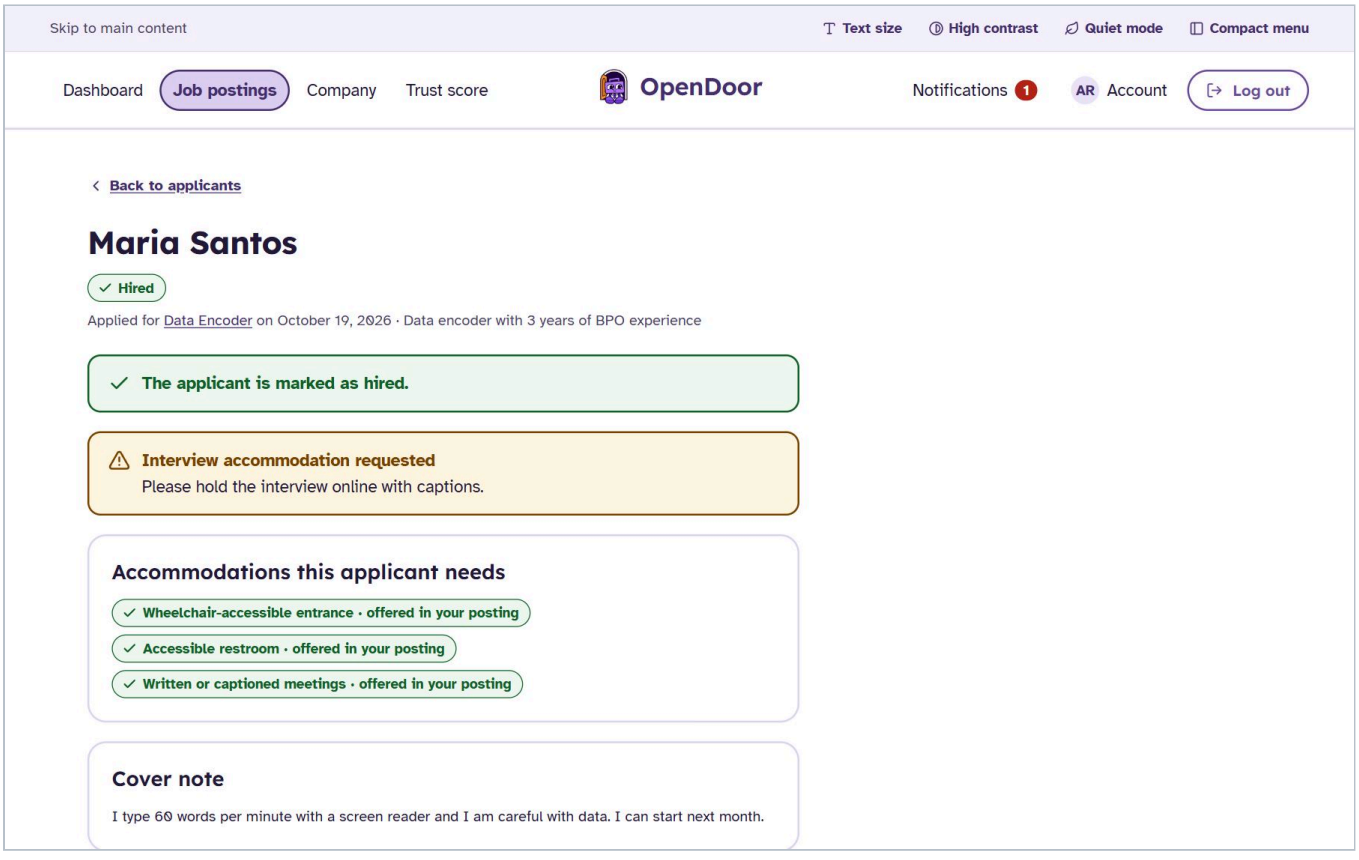
6.6 Mark the applicant as hired

Employer

In the UI: Press **Mark as hired** and confirm in the dialog.



The confirmation dialog



After Mark as hired

WHAT HAPPENS BEHIND THE SCREEN

- `PATCH /api/applications/{id}/status` with `{"status":"hired"}`. Core checks the allowed status workflow (hired and not selected are final).
- Core records which accommodations the posting promised (`application_accommodation`). After 30 days the candidate rates each one – that feedback builds the employer's trust score.

REQUEST (DEVTOOLS → NETWORK → HEADERS / PAYLOAD)

```
PATCH http://localhost:8080/api/applications/7/status
Status: 200
X-Request-ID: 85d518c4-a70c-44c5-b6f8-80ae8786d92f
Authorization: Bearer 2|•••••• (hidden)

{
  "status": "hired"
}
```

RESPONSE (DEVTOOLS → RESPONSE)

```
{
  "id": 7,
  "job_posting_id": 1,
  "job_posting": {
    "id": 1,
    "title": "Data Encoder",
    "description": "Encode and check customer records in our CRM.\nMonday to Friday, 9:00 to 17:00. Training is provid
ed.",
    "employer": {
      "id": 1,
      "company_name": "Bayanihan Tech Solutions",
      "verified": true
    },
    "category": {
      "id": 1,
      "name": "Administrative"
    },
    "address": "25 Scout Tobias St., South Triangle",
    "location": {
      "psgc_code": "1381300000",
      "label": "Quezon City, National Capital Region (NCR)"
    },
    "employment_type": "full_time",
    "work_setup": "onsite",
    "interview_format": "online",
    "status": "open",
    "closes_at": "2026-12-03",
    "accommodations": [
      {
        "id": 1,
        "name": "Wheelchair-accessible entrance",
        "group": "physical_access",
        "description": "Step-free entrance with a ramp or level access.",
        "is_retired": false,
        "note": "Ramp at the main entrance on Scout Tobias St."
… (59 more lines)
```

SERVICE LOGS (THE TERMINALS)

Gateway terminal (one JSON line per request, no headers or bodies)

```
{"time":"2026-10-19T15:39:26.567Z","request_id":"85d518c4-a70c-44c5-b6f8-80ae8786d92f","method":"PATCH","path":"/api/application
s/7/status","status":200,"duration_ms":211}
```

Core terminal

```
[PATCH] URI: /api/applications/7/status
```

DATABASE

Status `reviewed → hired`, `hired_at` and `hire_confirmed_by` filled, the promised accommodations saved, and a notification for the candidate.

Before

```
SELECT id, job_posting_id, candidate_profile_id, status, reviewed_at, hired_at, hire_confirmed_by, created_at FROM applications WHERE candidate_profile_id = 1 AND job_posting_id = 1
```

id	job_posting_id	candidate_profile_id	status	reviewed_at	hired_at	hire_confirmed_by	created_at
7	1	1	reviewed	2026-10-19 23:39:24	NULL	NULL	2026-10-19 23:39:21

After – application

```
SELECT id, job_posting_id, candidate_profile_id, status, reviewed_at, hired_at, hire_confirmed_by, created_at FROM applications WHERE candidate_profile_id = 1 AND job_posting_id = 1
```

id	job_posting_id	candidate_profile_id	status	reviewed_at	hired_at	hire_confirmed_by	created_at
7	1	1	hired	2026-10-19 23:39:24	2026-10-19 23:39:26	employer	2026-10-19 23:39:21

After – accommodations

```
SELECT application_id, accommodation_id FROM application_accommodation WHERE application_id = 7
```

application_id	accommodation_id
7	1
7	2
7	6
7	7

After – notifications

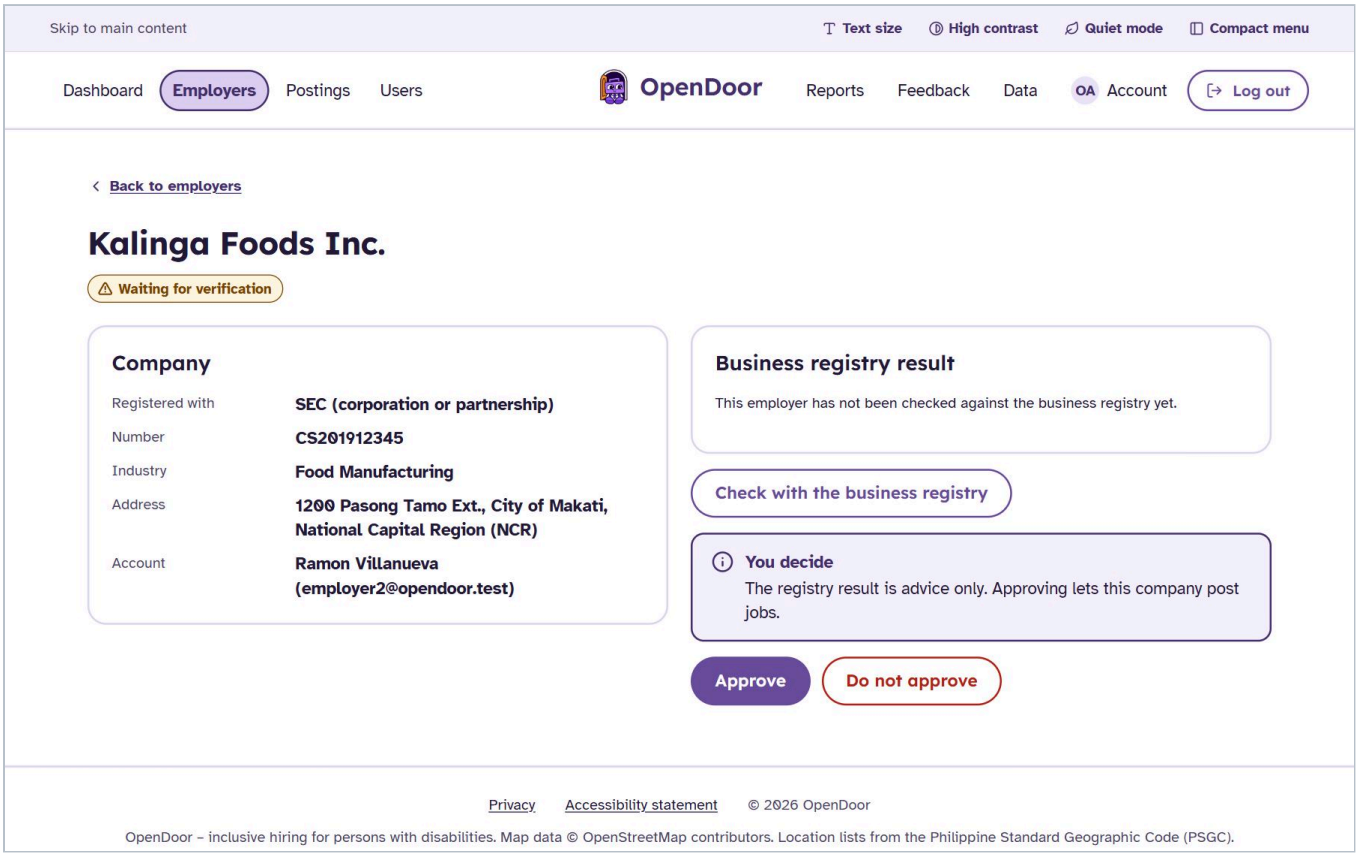
```
SELECT notifiable_id AS user_id, type, JSON_UNQUOTE(JSON_EXTRACT(data, '$.message')) AS message, created_at FROM notifications ORDER BY created_at DESC LIMIT 1
```

user_id	type	message	created_at
2	application_status_changed	Congratulations! Bayanihan Tech Solutions marked you as hired for Data Encoder. You can…	2026-10-19 23:39:26

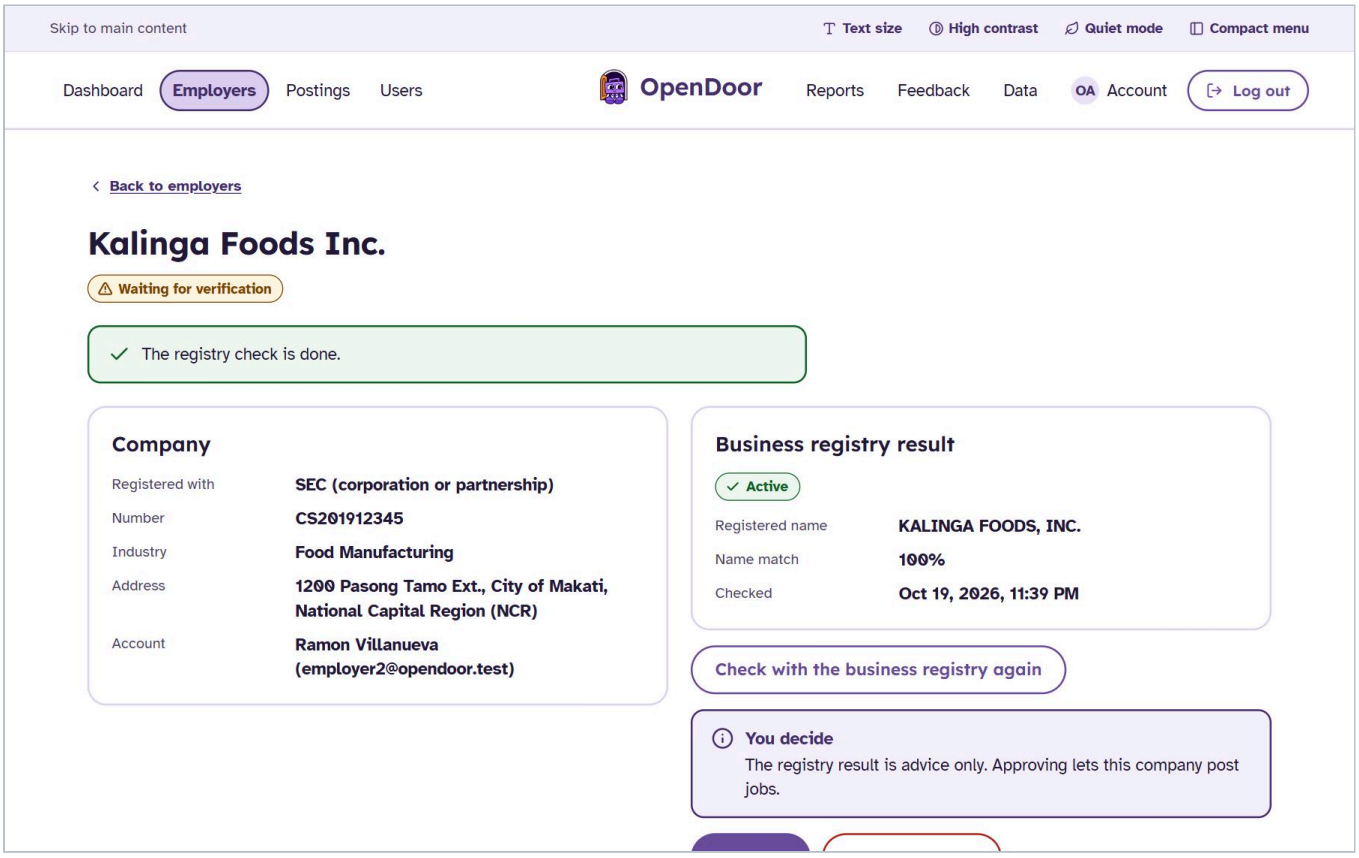
6.7 Check the employer with the business registry (SOAP)

Admin – admin@opendoor.test

In the UI: As the admin, open **Employers → Kalinga Foods Inc. → Check with the business registry**.



The employer waiting for verification



The registry result after the check

WHAT HAPPENS BEHIND THE SCREEN

- `PATCH /api/admin/employers/{id}/verify` with `{"action":"check"}` (admins only).
- Core's `BusinessRegistryClient` calls the legacy SOAP 1.1 service `VerifyBusinessRegistration` with a WS-Security UsernameToken, 5 s timeout. The registry looks the number up in its own SQLite data (a simulated DTI/SEC registry).
- Core stores the answer, including the raw XML, in `registry_checks`. A SOAP fault or a timeout becomes a clear error message; the admin can still decide by hand.

REQUEST (DEVTOOLS → NETWORK → HEADERS / PAYLOAD)

```
PATCH http://localhost:8080/api/admin/employers/2/verify
Status: 200
X-Request-ID: c98d1ca9-1957-4e4e-ae9e-fd84c0b93e02
Authorization: Bearer 3|•••••• (hidden)

{
  "action": "check"
}
```

RESPONSE (DEVTOOLS → RESPONSE)

```
{
  "registry": {
    "status": "ACTIVE",
    "registered_name": "KALINGA FOODS, INC.",
    "name_match": 1,
    "checked_at": "2026-10-19T23:39:30+08:00"
  },
  "verified_at": null
}
```

Also on this screen: `GET /api/me`, `GET /api/admin/employers/2` (page data: user, notifications, lists).

THE SOAP MESSAGES BETWEEN CORE AND THE REGISTRY

Not visible in the browser – Core sends them server to server. Same envelope as Core's client; the WS-Security user name and password are hidden here.

```
<?xml version="1.0" encoding="UTF-8"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/" xmlns:ns1="urn:opendoor:registry:v1" xmlns:ns2
="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd">
  <SOAP-ENV:Header>
    <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd">
      <wsse:UsernameToken>
        <wsse:Username>•••••• (hidden)</wsse:Username>
        <wsse:Password Type="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-username-token-profile-1.0#PasswordTex
t">•••••• (hidden)</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
  </SOAP-ENV:Header>
  <SOAP-ENV:Body>
    <ns1:VerifyBusinessRegistrationRequest>
      <ns1:RegistrationNumber>CS201912345</ns1:RegistrationNumber>
      <ns1:RegistryType>SEC</ns1:RegistryType>
      <ns1:BusinessName>Kalinga Foods Inc.</ns1:BusinessName>
    </ns1:VerifyBusinessRegistrationRequest>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/" xmlns:ns1="urn:opendoor:registry:v1">
  <SOAP-ENV:Body>
    <ns1:VerifyBusinessRegistrationResponse>
      <ns1:Found>true</ns1:Found>
      <ns1:Status>ACTIVE</ns1:Status>
      <ns1:RegisteredName>KALINGA FOODS, INC.</ns1:RegisteredName>
      <ns1:DateRegistered>2019-03-14</ns1:DateRegistered>
      <ns1:NameMatchScore>1</ns1:NameMatchScore>
      <ns1:CheckedAt>2026-10-19T23:39:32+08:00</ns1:CheckedAt>
```

```
    </ns1:VerifyBusinessRegistrationResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

SERVICE LOGS (THE TERMINALS)

Gateway terminal (one JSON line per request, no headers or bodies)


```
{"time":"2026-10-19T15:39:30.863Z","request_id":"c98d1ca9-1957-4e4e-ae9e-fd84c0b93e02","method":"PATCH","path":"/api/admin/employers/2/verify","status":200,"duration_ms":183}
```


Core terminal

```
[PATCH] URI: /api/admin/employers/2/verify
```

Business Registry terminal

```
[200]: POST /soap
```

DATABASE

A new `registry_checks` row with the registry's status, registered name and name-match score.

Before

```
SELECT id, employer_id, business_reg_no, registry_type, status, registered_name, name_match_score, checked_at FROM registry_checks WHERE employer_id = 2 ORDER BY id DESC LIMIT 1
```

id	employer_id	business_reg_no	registry_type	status	registered_name	name_match_score	checked_at
(no rows)							

After

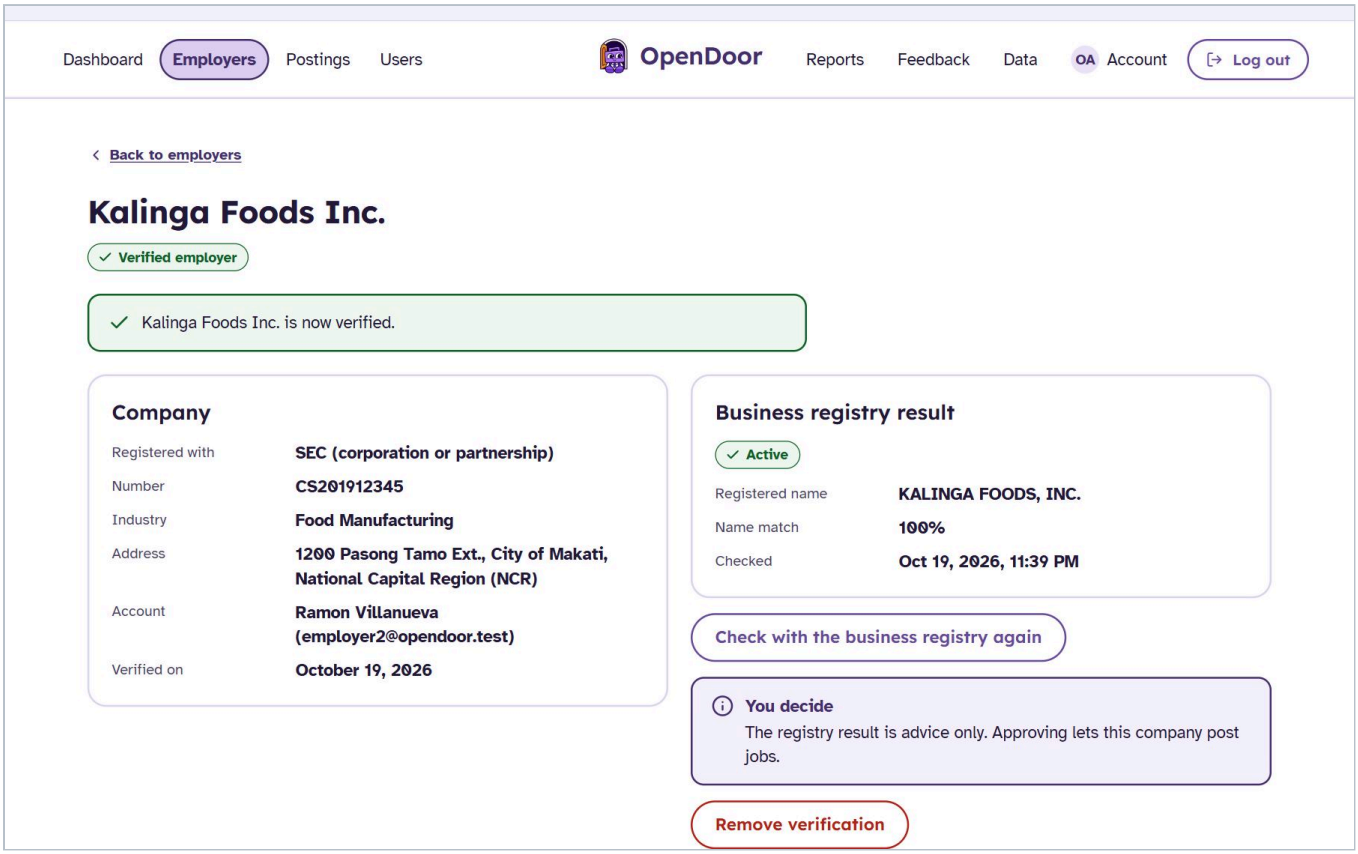
```
SELECT id, employer_id, business_reg_no, registry_type, status, registered_name, name_match_score, checked_at FROM registry_checks WHERE employer_id = 2 ORDER BY id DESC LIMIT 1
```

id	employer_id	business_reg_no	registry_type	status	registered_name	name_match_score	checked_at
1	2	CS201912345	SEC	ACTIVE	KALINGA FOODS, INC.	1.00	2026-10-19 23:39:30

6.8 Approve the employer

Admin

In the UI: Press **Approve**.



After Approve

WHAT HAPPENS BEHIND THE SCREEN

- Same endpoint with `{"action":"approve"}`. The employer becomes verified (badge on postings and the public page) and is notified.

REQUEST (DEVTOOLS → NETWORK → HEADERS / PAYLOAD)

```
PATCH http://localhost:8080/api/admin/employers/2/verify
Status: 200
X-Request-ID: 7487995e-8186-49f2-be42-d18b036dc950
Authorization: Bearer 3|•••••• (hidden)

{
  "action": "approve"
}
```

RESPONSE (DEVTOOLS → RESPONSE)

```
{
  "registry": {
    "status": "ACTIVE",
    "registered_name": "KALINGA FOODS, INC.",
    "name_match": 1,
    "checked_at": "2026-10-19T23:39:30+08:00"
  },
  "verified_at": "2026-10-19T23:39:32+08:00"
}
```

SERVICE LOGS (THE TERMINALS)

Gateway terminal (one JSON line per request, no headers or bodies)

{"time":"2026-10-19T15:39:32.557Z","request_id":"7487995e-8186-49f2-be42-d18b036dc950","method":"PATCH","path":"/api/admin/employers/2/verify","status":200,"duration_ms":182}

Core terminal

```
[PATCH] URI: /api/admin/employers/2/verify
```

Insights terminal

{"source":"overpass","server":"overpass-api.de","error":"ECONNABORTED"}

DATABASE

`employers.verified_at` filled, and an `employer_verification` notification for the employer.

Before

```
SELECT id, company_name, verified_at, verification_rejected_reason FROM employers WHERE id = 2
```

id	company_name	verified_at	verification_rejected_reason
2	Kalinga Foods Inc.	NULL	NULL

After – employer

```
SELECT id, company_name, verified_at, verification_rejected_reason FROM employers WHERE id = 2
```

id	company_name	verified_at	verification_rejected_reason
2	Kalinga Foods Inc.	2026-10-19 23:39:32	NULL

After – notifications

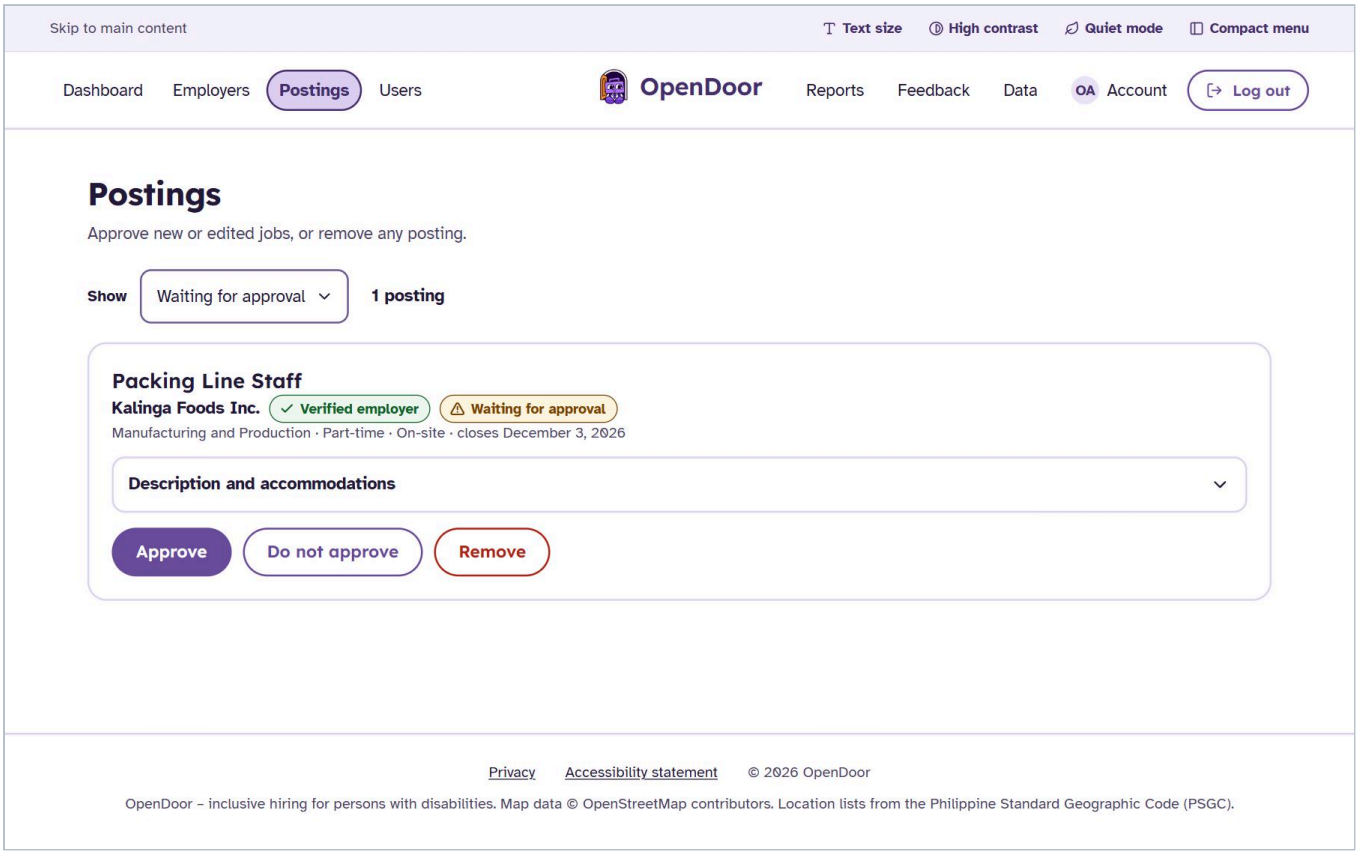
```
SELECT notifiable_id AS user_id, type, JSON_UNQUOTE(JSON_EXTRACT(data, '$.message')) AS message, created_at FROM notifications ORDER BY created_at DESC LIMIT 1
```

user_id	type	message	created_at
5	employer_verification	Kalinga Foods Inc. is now verified. Candidates see the Verified badge on your postings.	2026-10-19 23:39:32

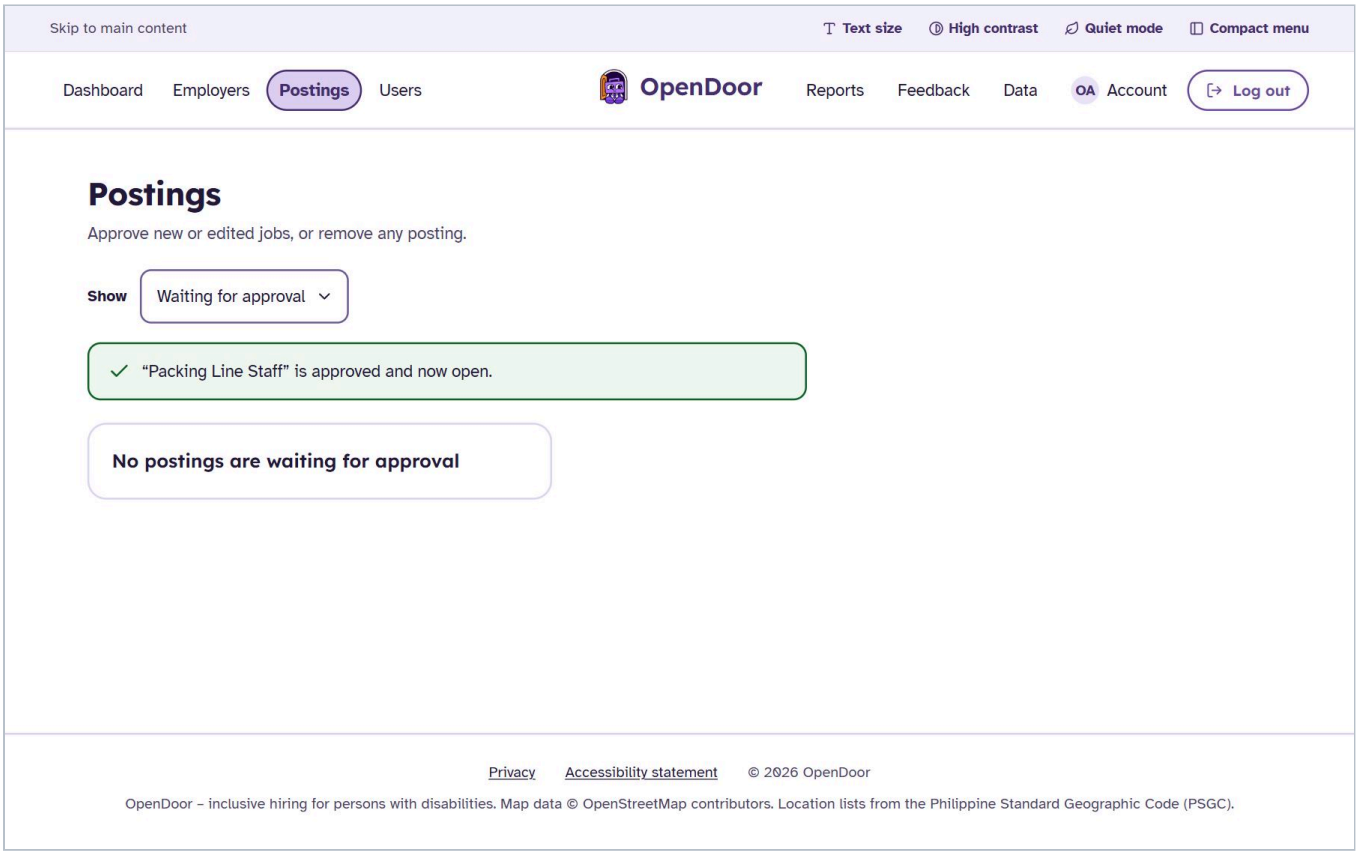
6.9 Approve a job posting

Admin

In the UI: Open **Postings** (waiting for approval) and press **Approve** next to Packing Line Staff.



Postings waiting for approval



After Approve

WHAT HAPPENS BEHIND THE SCREEN

- `PATCH /api/admin/job-postings/{id}/approve` with `{"action":"approve"}`. The page then reloads the queue.
- From now on the posting is in search results and can receive applications.

REQUEST (DEVTOOLS → NETWORK → HEADERS / PAYLOAD)

```
PATCH http://localhost:8080/api/admin/job-postings/4/approve
Status: 200
X-Request-ID: 2dad3457-63fa-4fc0-a3c9-63e9f2651a95
Authorization: Bearer 3|•••••• (hidden)

{
  "action": "approve"
}
```

RESPONSE (DEVTOOLS → RESPONSE)

```
{
  "id": 4,
  "title": "Packing Line Staff",
  "description": "Pack and label food products.\nSeated workstation available.",
  "employer": {
    "id": 2,
    "company_name": "Kalinga Foods Inc.",
    "verified": true
  },
  "category": {
    "id": 10,
    "name": "Manufacturing and Production"
  },
  "address": "1200 Pasong Tamo Ext.",
  "location": {
    "psgc_code": "1380300000",
    "label": "City of Makati, National Capital Region (NCR)"
  },
  "employment_type": "part_time",
  "work_setup": "onsite",
  "interview_format": "onsite",
  "status": "open",
  "closes_at": "2026-12-03",
  "accommodations": [
    {
      "id": 11,
      "name": "Part-time schedule",
      "group": "work_arrangement",
      "description": "Reduced weekly hours are available.",
      "is_retired": false,
      "note": null
    },
    {
      "id": 14,
… (17 more lines)
```

Also on this screen: `GET /api/me`, `GET /api/admin/job-postings?status=pending_approval&page=1` (page data: user, notifications, lists).

SERVICE LOGS (THE TERMINALS)

Gateway terminal (one JSON line per request, no headers or bodies)

```
{"time":"2026-10-19T15:39:34.658Z","request_id":"2dad3457-63fa-4fc0-a3c9-63e9f2651a95","method":"PATCH","path":"/api/admin/job-postings/4/approve","status":200,"duration_ms":126}
```

Core terminal

```
[PATCH] URI: /api/admin/job-postings/4/approve
```

DATABASE

Status `pending_approval → open`, `is_approved` 0 → 1, and a `posting_review` notification for the employer.

Before

```
SELECT id, title, status, is_approved, updated_at FROM job_postings WHERE id = 4
```

id	title	status	is_approved	updated_at
4	Packing Line Staff	pending_approval	0	2026-10-19 23:38:54

After – posting

```
SELECT id, title, status, is_approved, updated_at FROM job_postings WHERE id = 4
```

id	title	status	is_approved	updated_at
4	Packing Line Staff	open	1	2026-10-19 23:39:34

After – notifications

```
SELECT notifiable_id AS user_id, type, JSON_UNQUOTE(JSON_EXTRACT(data, '$.message')) AS message, created_at FROM
notifications ORDER BY created_at DESC LIMIT 1
```

user_id	type	message	created_at
5	posting_review	Your posting Packing Line Staff was approved and is now open to candidates.	2026-10-19 23:39:34

7 Validation script

`tests/validate/validate_opendoor.py` checks a running stack in a few seconds: every service, the gateway's routing and the service-to-service call. It sends only `GET` requests and changes no data, so it is safe on production. Python 3.8+, standard library only.

Run it

```
python tests/validate/validate_opendoor.py                    # local stack (all 14 checks)
python tests/validate/validate_opendoor.py --timeout 5        # slower machine
python tests/validate/validate_opendoor.py --gateway https://gateway-production-4e58.up.railway.app --gateway-only   #
production (7 checks)
```

Exit code 0 = all passed, 1 = something failed. Options: `--id 3` (sample posting), `--gateway/--core/--insights/--registry URL`, `--snapshot-timeout` (default 15 s).

What it checks

Check	Proves
Core, Insights, registry and its WSDL are up	Each service runs; the registry has its data
Gateway `/api/health` shows core and insights `up`	The gateway's `CORE_URL` / `INSIGHTS_URL` are right
A sample posting from the search list	There is data to test with
Core returns it directly, and the same through the gateway	The gateway forwards `/api/*` to Core
Unknown posting → 404 `NOT_FOUND` with `request_id`	The error envelope passes through the gateway
`/api/me` without a token → 401	Protected routes need a login
Insights snapshot with `sources.core = ok`, directly and through the gateway	Service-to-service: Insights reached Core's `/internal` route with the service key
Core `/internal` without the key → 401; gateway `/internal` → 404	Internal routes are protected and never public

Sample output (local stack, October 19, 2026)

```
[PASS] Core is up (GET /up)
[PASS] Insights is up (GET /health)
[PASS] Business Registry is up (GET /health, database ok)
[PASS] Business Registry serves its WSDL (GET /soap?wsdl)
[PASS] Gateway reaches Core and Insights (GET /api/health)
[PASS] Sample posting found (GET /api/job-postings)
       sample job posting id = 4
[PASS] Core returns the posting directly (GET /api/job-postings/{id})
[PASS] Gateway routes /api/* to Core: same posting
[PASS] Gateway passes Core's 404 for an unknown posting (error envelope)
[PASS] Gateway requires a token for /api/me (401)
[PASS] Insights builds the snapshot from Core's /internal summary
       sources = {'core': 'ok', 'psgc': 'cached', 'nominatim': 'unavailable', 'overpass': 'unavailable'}
[PASS] Gateway routes /api/insights/* to Insights: same snapshot
       sources = {'core': 'ok', 'psgc': 'cached', 'nominatim': 'unavailable', 'overpass': 'unavailable'}
[PASS] Core refuses /internal without the service key (401)
[PASS] Gateway never exposes /internal (404)

Result: 14/14 checks passed
```

`psgc`, `nominatim` and `overpass` may be `unavailable` when the public servers are busy – the designed fallback, not a failure. Only `core` must be `ok`. The script's README (`tests/validate/README.md`) lists the common causes when a check fails.

8 Testing tools

Tool	Folder	Command	What it checks	Last result
PHPUnit (Core)	`services/core-api`	`php artisan test`	Auth, roles and policies, every module's endpoints, status workflow, matching, trust score, SOAP client (faults, timeouts), error envelope	372 tests, 1,776 assertions passed (Oct 19, 2026)
Jest (gateway)	`gateway`	`npm test`	Routing, CORS, rate limit, request IDs, timeouts and the error envelope when a service is down	24 passed (Oct 19, 2026)
Jest (Insights)	`services/insights-service`	`npm test`	Snapshot building, caching, budgets and fallbacks when PSGC, Nominatim or Overpass fail	135 passed (Oct 19, 2026)
Registry runner	`services/business-registry-soap`	`php tests/run.php`	Lookups, name matching, WS-Security and SOAP faults, plus a smoke test on its own port	39/39 checks passed (Oct 19, 2026)
Vitest (React)	`frontend`	`npm test` (and `npm run lint`)	Pages and components: forms, validation messages, role menus, accessibility behaviour	165 passed (Oct 19, 2026)
Postman / Newman	repo root	`npx newman run tests/postman/OpenDoor.postman_collection.json -e tests/postman/local.postman_environment.json --env-var "demoPassword=…"`	The SIA test plan (T01–T16) end to end through the gateway: 199 requests, every response checked for status, body, `X-Request-ID` and time	CI: 193 requests / 933 assertions / 0 failures (collection before onboarding); production smoke 73 / 389 / 0 (Oct 16, 2026)
JMeter	`tests/jmeter`	`jmeter -n -t opendoor-matching.jmx -Jhost=<gateway host> -l results.jtl -e -o report/run`	Load (50 users) and stress (100/200 users) on the matching search; needs test data from `php artisan load-test:data`	Production, 50 users: P95 204 ms, 0 % errors (Oct 16, 2026) – SIA doc 7.3
axe-core / Lighthouse	`tests/a11y`	`npm run axe` with `APP_URL` (built app, `npm run preview` on 4173), `API_URL`, `DEMO_PASSWORD`	WCAG 2.1 A/AA rules on the main pages of every role	axe: 0 violations on 33 pages (CI); Lighthouse 100 on 6 guest pages
Validation script	repo root	`python tests/validate/validate_opendoor.py`	14 quick checks of a running stack (section 7)	14/14 local (Oct 19, 2026)

Tool	Folder	Command	What it checks	Last result
GitHub Actions CI	`.github/workflows/ci.yml`	runs on every push / pull request	All of the above per changed component, then the whole stack in Docker with Newman and axe	All 7 jobs green (run 38493807232)

- Unit tests need no running services (Core uses an in-memory SQLite database). Newman, axe and the validation script need the running stack.
- Newman's full collection creates and deletes test data; run it locally or in CI only. Production gets the read-only smoke collection (`OpenDoor.smoke.postman_collection.json`). Details: `tests/postman/README.md`, `tests/jmeter/README.md`.
- Results of the last full runs (CI, production smoke, JMeter) are in the SIA documentation, sections 7.1–7.3.

9 Troubleshooting

Problem	Likely cause	Fix
A start command says the port is already in use	An old terminal or a crashed run still holds it	Find and stop the process (see 3.4), or close the old terminal.
The app shows "Something went wrong" or every page fails to load	The gateway is not running, or Core behind it is down	Open `http://localhost:8080/api/health`: `core: down` → start Core; no answer → start the gateway.
Log in fails with a network error in the browser console (CORS)	The app was opened on another address than the gateway allows	Open `http://localhost:5173` (not 127.0.0.1); the gateway allows the origin in `CORS_ALLOWED_ORIGIN`.
Log in says the email or password is wrong	The database was seeded with another `DEMO_PASSWORD`	Use the password the database was seeded with (ask the team), or re-seed: `php artisan migrate:fresh --seed` (deletes local data).
Core errors with "SQLSTATE … Connection refused"	MySQL is not running	`Start-Service MySQL80` in an admin PowerShell.
Registry check says the registry is unavailable	The SOAP registry (8003) is not running or was started on `localhost`	Start it with `php -S 127.0.0.1:8003 -t public` (on Windows `localhost` may bind IPv6 only).
The snapshot shows "not available" or no nearby places	Overpass or Nominatim (public servers) are busy or offline – the designed fallback	Open the job page again after a minute (Insights retries and caches). Before a presentation, open each job page once so the answers are cached.
In `npm run dev` only: the snapshot of a place that is not cached yet shows "not available" after ~20 s	React development mode sends the request twice; Chrome waits for the first identical request, so the second one runs out of time when Overpass is slow	Open the page again once the first answer is cached, or present from the built app (Vercel or `npm run build` + `npm run preview`).
Forgot-password email never arrives (local)	Reset emails are queued; locally mail goes to the log	Run `php artisan queue:work database --stop-when-empty` in `services/core-api`; the link is in `storage/logs/laravel.log`.
Production is slow on the first request	Railway services were idle	Open `/api/health` on the gateway a few minutes before presenting.
Newman: a few requests over 1000 ms locally	`php artisan serve` handles one request at a time	Run again; production and CI are the reference for timing.

Any error the API returns carries a `request_id`. Search for it in the gateway terminal (and Railway's logs) to find the request and the service that answered.

A Rebuilding this guide

- Texts: `tests/a11y/guides/run-guide-content.mjs`. Builder: `build-run-guide.mjs`. Walkthrough capture: `capture-walkthrough.mjs` (same folder).
- Capture: start the stack on a freshly seeded copy of the database (for example `opendoor_core_test`), with each service's output written to a log file, then in `tests/a11y`: `DEMO_PASSWORD=… LOG_DIR=<log folder> node guides/capture-walkthrough.mjs`.
- Build: `node guides/build-run-guide.mjs` → `docs/guides/OpenDoor-Run-and-Demo-Guide.pdf` (no passwords).
- Copy with passwords (not committed): set `PROD_DEMO_PASSWORD` and/or `LOCAL_DEMO_PASSWORD` and run the build again → `docs/guides/private/OpenDoor-Run-and-Demo-Guide-with-passwords.pdf` (git-ignored).